This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Technical Proposal - Smart Health Checkup Recommendation Dashboard

1. Executive Summary
   - 1.1 Introduction
   - 1.2 Background Information
   - 1.3 To-Be Vision & Core Value Proposition
2. Project Scope & Assumptions
   - 2.1 Scope of Work (In-Scope Deliverables)
   - 2.2 Out of Scope
   - 2.3 Key Assumptions & Dependencies
3. High-Level System Architecture
   - 3.1 Data flow
   - 3.2 Conceptual Architecture
   - 3.3 Core Modules Overview
4. Data Platform & Integration Architecture
   - 4.1 High-level Data Platform Architecture
   - 4.2 Data Pipeline at a Glance
   - 4.3 Data Ingestion Layer
   - 4.4 Storage & Processing Layer
   - 4.5 Dual-Storage Strategy
   - 5. Data Platform Tech Stack
6. Security, Privacy & Data Governance
   - 6.1 SMART on FHIR Authorization (Access Control Gatekeeper)
   - 6.2 Data Masking & Pseudo-Anonymization Pipelines
   - 6.3 Comprehensive Audit Logging
7. AI & Recommendation Engine Architecture
   - 7.1 High-Level AI Workflow
   - 7.2 Model Context Protocol (MCP) Middleware (Secure Context Retrieval)
   - 7.3 Hybrid Recommendation Logic (Rule-based Safety + AI Inference)
   - 7.4 Central AI Gateway
   - 7.5 AI Tech Stack
8. Frontend & Dashboards (User Experience)
   - 8.1 Responsive Web Application Architecture
   - 8.2 Business Intelligence & Conversion Dashboard (TOR §7)
9. Deployment & Operational Architecture
   - 9.1 On-Premise Deployment Strategy
   - 9.2 Observability & Monitoring (OSS Stack)
   - 9.3 Disaster Recovery (DR) & Backup Plan
10. Quality Assurance & Non-Functional Requirements (NFRs)
    - 10.1 Performance Metrics & SLAs
    - 10.2 Scalability & High Availability Matrix
    - 10.3 Medical Validation Strategy (Medical Acceptance Test)
    - 10.4 Safety "Fallback" Rules
11. Infrastructure Licensing & Cloud Costs
    - 11.1 Workload Assumptions
    - 11.2 Software Licensing Costs
    - 11.3 External AI API Costs (Gemini 1.5 Pro - example)
    - 11.4 Infrastructure Cost Comparison: On-Premise vs. AWS Cloud
    - 11.5 Three-Year Total Cost of Ownership (TCO) Summary
    - 11.6 Comparison Summary & Recommendation

1. Executive Summary

1.1 Introduction

This technical proposal outlines the design, architecture, and implementation strategy for the **Smart Health Checkup Recommendation Dashboard** at Navavej Hospital. The primary objective is to develop an intelligent, AI-driven platform that systematically analyzes a patient's historical medical data to recommend highly accurate, safe, and personalized health checkup programs. By combining international healthcare data standards (HL7 FHIR R4) with an advanced, pluggable Artificial Intelligence architecture (Model-Switching Architecture), the proposed system will elevate Navavej Hospital's services to a true "Personalized Medicine" approach. The solution inherently prioritizes security and privacy, utilizing strict Data Masking, SMART on FHIR authorization, and the Model Context Protocol (MCP) to ensure full compliance with PDPA and HIPAA regulations.

1.2 Background Information

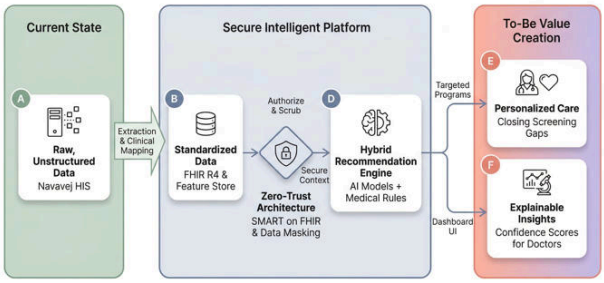
Traditionally, health checkup programs are selected based on generic criteria such as age and gender. However, this generalized approach frequently leads to "Screening Gaps"—missed opportunities to identify hidden health risks or prioritize critical preventative care. Service recipients possess rich, longitudinal medical histories stored within the Navavej Hospital Information System (HIS / IQVIA SaaS), encompassing diagnoses, lab value velocities, biomarker trends (e.g., eGFR slope or BMI), and historical screening data.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

To bridge the gap between this wealth of raw data and actionable clinical insights, Navavej requires a system capable of interpreting complex medical histories efficiently and securely. This entails moving away from manual, ad-hoc program selection toward a **Hybrid Recommendation Engine (Rule-based combined with AI)**. The core challenge to realizing this vision is not only generating medically accurate recommendations but also standardizing the legacy data using international clinical code mapping (LOINC, SNOMED CT, ICD-10) and strictly safeguarding patient privacy before any data interacts with external AI models. This proposal presents a modular, highly secure technical blueprint designed specifically to leapfrog these challenges, while supporting seamless future AI upgrades without core system disruptions.

1.3 To-Be Vision & Core Value Proposition



The "To-Be" vision is a fully automated, transparent, and standardized workflow where raw clinical data transforms into highly personalized patient care without compromising a single data privacy boundary. The core value propositions of this platform are:

(XXXXX) We should avoid the word “To-be” vision and “To-be Value creation” basically, we should not add anything related to To-be or add-on something that is our idea and not in TOR yet (We should have separated section for that)

1. **Precision and Personalization:** The system will evaluate dozens of hard clinical markers (e.g., BMI velocity, JNC 8 blood pressure categories, genetic histories) to generate customized primary and "Add-On" examination recommendations that successfully close vital Screening Gaps.
2. **Explainable AI (XAI) for Trust:** Both healthcare providers and service recipients will receive clear, logical reasoning behind every AI recommendation alongside a Confidence Score, completely eliminating "black-box" medical uncertainty.
3. **Zero-Trust Security by Design:** Applying the SMART on FHIR authorization standard and the strict Least Privilege Model Context Protocol (MCP) ensures that PII is masked before any inference occurs, and every AI prompt is immutably logged for comprehensive auditing.
4. **Future-Proof Pluggable Architecture:** Exploring a Model-Switching AI Gateway guarantees Navavej Hospital flexibility. Administrators can effortlessly transition between frontier AI models (e.g., from GPT-4 to Claude 3.5 or an on-premise Local Llama model) via configuration screens, entirely avoiding vendor lock-in as technology advances.

2. Project Scope & Assumptions

2.1 Scope of Work (In-Scope Deliverables)

The implementation of the Smart Health Checkup Recommendation Dashboard encompasses the complete end-to-end delivery of the data, security, AI, and presentation layers.

Implementation Layer	Deliverables & Included Scope
Data Integration & Storage	• Secure data extraction from Navavej HIS (IQVIA SaaS) via CDC sync. • Automated processing and terminology mapping into international clinical codes (LOINC, SNOMED CT, ICD-10). • Setup of the dual-storage architecture: Core **FHIR Database** & analytical **Trending Store DB**.
AI Recommendation Engine	• Development of the **Hybrid Recommendation Engine** combining hard medical rules with AI logic. • Delivery of the **AI Gateway** supporting pluggable, model-switching architecture. • Implementation of the **Model Context Protocol (MCP) Server** for secure context retrieval.
Security & Compliance	• **SMART on FHIR** authorization server acting as the gateway for all FHIR data. • Strict **Data Masking** protocol within the MCP layer to eliminate PII leakage to the AI. • Implementation of the immutable **5-Dimension Audit Log** (Who, When, Where, What, Why).

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

User Interfaces (Web)	• **Healthcare Provider Dashboard:** Screens displaying Individual Summaries, historical Trend Timelines (3-5 years), Screening Gaps, and Explainable AI reasoning. • **Admin/DPO Dashboard:** Screens for Real-Time Access Monitoring and Behavior Anomaly Detection. • **Management team:** Conversion tracking and business performance dashboard.
Testing & Deployment	• Complete system containerization (Docker/Microservices). • **Medical Acceptance Testing (MAT)** to ensure the AI hits the > 95% threshold of expert consensus. • Performance testing to ensure strict SLA compliance (< 10-second latency).

2.2 Out of Scope

To ensure strict project boundaries and focus, the following items are excluded from this implementation phase:

Out-of-Scope Area	Description
Legacy HIS Replacement	The proposed solution acts as an intelligent add-on. It does not replace, refactor, or migrate the core functionality of the existing Navavej Hospital Information System (IQVIA).
Bi-Directional EMR Writes	The AI operates purely in a read-only state via the MCP server to deduce recommendations. It will not autonomously write back to or alter the core medical records in the HIS.
Financial / Billing AI	The AI scope is strictly limited to medical health checkup recommendations. Predictive models for hospital financial forecasting or billing are excluded.
Physical Hardware	Procurement, delivery, or maintenance of physical on-premise hardware servers is excluded (presuming standard cloud infrastructure usage).

2.3 Key Assumptions & Dependencies

The successful delivery of this architecture relies on the following baseline assumptions:

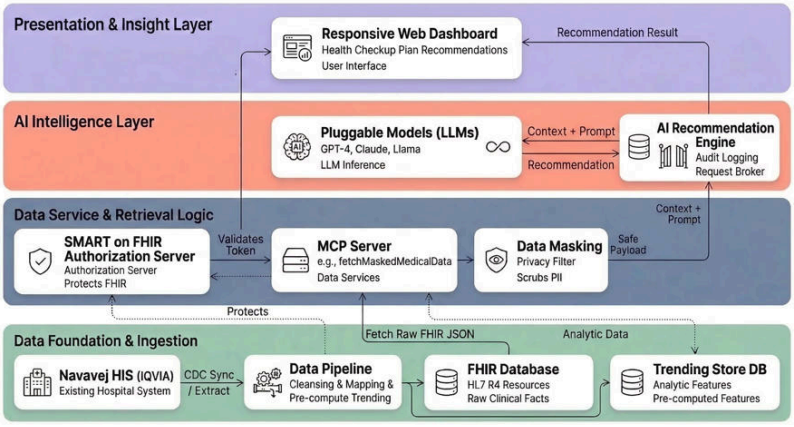
Dependency Domain	Assumption Statement
Data Availability	The existing Navavej HIS (IQVIA SaaS) supports and permits reliable, near-real-time data synchronization (e.g., CDC or robust APIs) necessary to populate the pipeline.
Medical Ruleset Support	Clinical experts at Navavej will provide the definitive "Safety Rules" (e.g., age limits, gender-specific checks) required to train and constrain the rule-based safety engine.
LLM Infrastructure	Commercial LLM API accounts (such as OpenAI/Anthropic) or provisioned local GPU hardware (if utilizing a localized LLM) will be funded or provided by the hospital.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

3. High-Level System Architecture

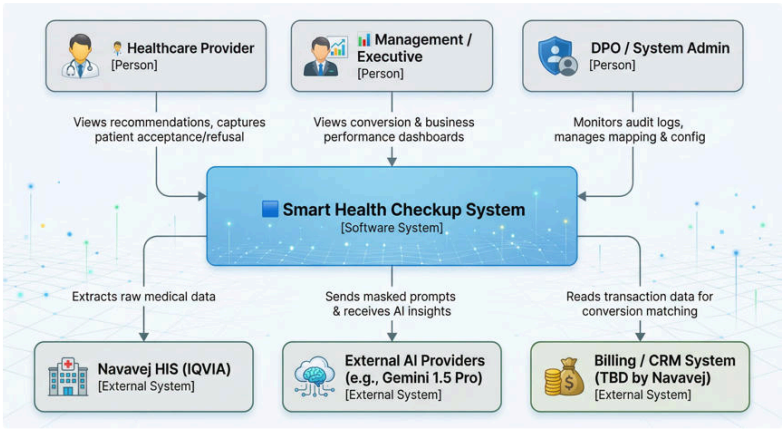
3.1 Data flow



3.2 Conceptual Architecture

C4 Context Diagram

This diagram maps out the high-level interactions between the users, the proposed system, and the external legacy systems. (XXXXX) We should add another stakeholder (Management team) to check conversion dashboard



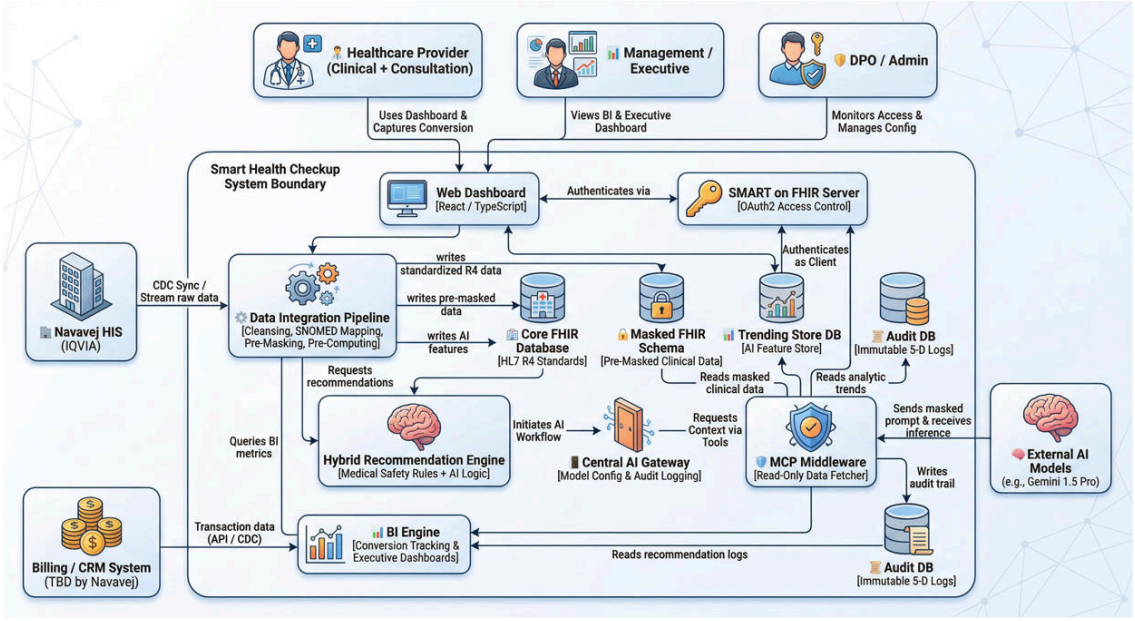
› mermaid

C4 Container Diagram

This diagram zooms inside the system boundary to show the high-level technical architecture, demonstrating how the Pluggable AI, dual-databases, and MCP Middleware interact safely.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon



mermaid

3.3 Core Modules Overview

To realize the "To-Be" architecture, the system is logically divided into 7 specialized, decoupled microservice modules:

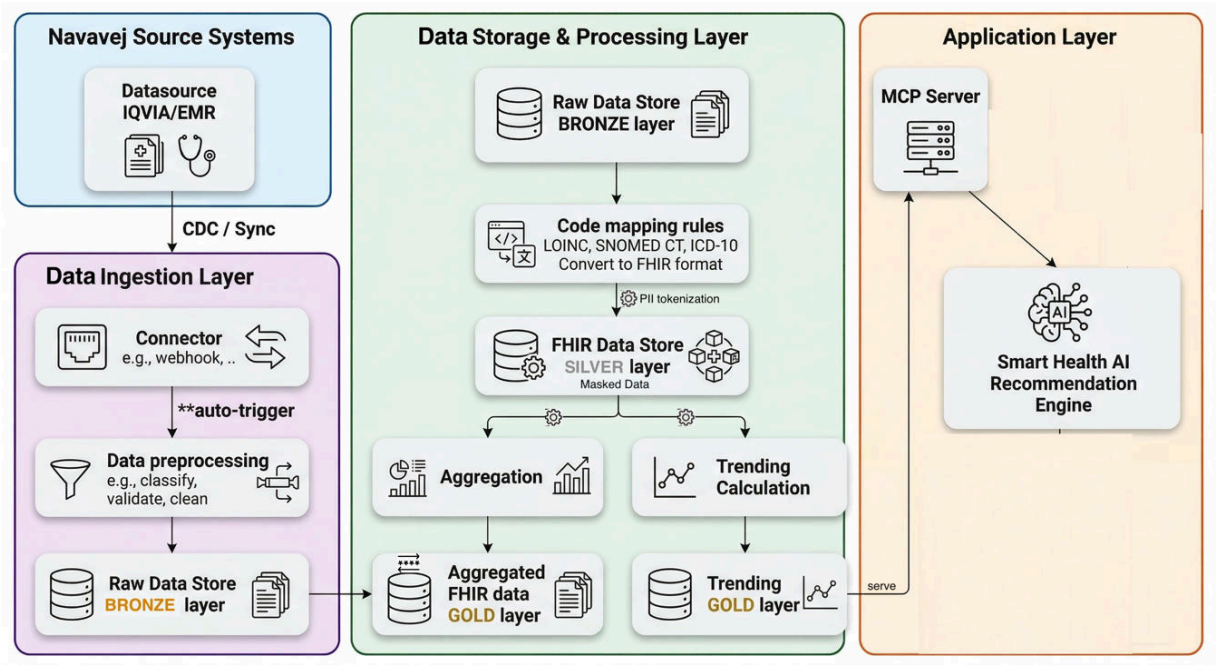
1. **Web Dashboard (Frontend UI):** A responsive application (e.g., React/Next.js) serving 2 primary user groups. It provides:
   - Sales/Doctors: with the **Individual Summary** and **Trend Timelines**
   - DPO: with an **Access Monitoring** page for anomaly detection.
2. **Data Integration Pipeline:** A robust ETL engine that captures updates from the Navavej HIS via CDC sync. It handles data cleansing, strict clinical code mapping (LOINC, SNOMED CT, ICD-10), and the pre-computation of complex health trends (e.g., eGFR slope or BMI velocity).
3. **Dual-Storage Databases:** The data storage layer is deliberately split. The **Core FHIR DB** maintains immutable, standardized clinical records, while the **Trending Store DB** acts as a flexible AI Feature Store for pre-computed longitudinal data.
4. **SMART on FHIR Authorization Server:** The centralized OAuth 2.0 gatekeeper. It ensures that every request pulling from the databases—whether requested by a doctor's dashboard or the MCP context fetcher—operates under strict, resource-level scopes.
5. **Hybrid Recommendation Engine:** The orchestrator of the system's clinical logic. It applies deterministic "Safety Rules" (e.g., specific age/gender limits) *before and after* invoking the AI, ensuring medical safety acts as a fail-safe parachute alongside the statistical AI model.
6. **Model Context Protocol (MCP) Middleware:** A specialized, Read-Only data fetcher. It serves as the ultimate privacy filter, pulling records from the FHIR and Trending databases and strictly scrubbing all Personally Identifiable Information (PII) before the context is ever shared with the AI.
7. **AI Gateway & Pluggable Models:** A model-agnostic API router that bridges the masked context to frontier LLMs (e.g., GPT-4, Claude 3.5, Local Llama). Crucially, this module is also responsible for executing the immutable 5-Dimension Audit Logging (Who, When, Where, What, Why) for every single AI inference execution.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

4. Data Platform & Integration Architecture

4.1 High-level Data Platform Architecture



› mermaid

The High-Level Data Platform operates on a highly decoupled, event-driven Medallion Lakehouse pattern. It systematically transforms raw clinical records into secure, AI-ready features without compromising performance or privacy. The architecture is logically partitioned into four primary boundaries:

- **Source Systems (Navavej Source Systems)** The origin of all clinical telemetry. This boundary represents the existing Navavej infrastructure, primarily the Navavej EMR, which captures patient data across four core categories (Demographics, Conditions, Lab Results, etc.). This data is systematically synchronized into the IQVIA (SaaS) operational store, which serves as the primary extraction point for our platform.
- **Ingestion Layer** The secure, event-driven entry point into the data platform. Utilizing Change Data Capture (CDC) streams from IQVIA, this layer continuously evaluates incoming payloads. It employs deterministic routing logic (e.g., detecting "New Patient" status to execute ID tagging prior to appending EMR records). Additionally, it isolates Thai HIS-specific local codes, routing them through dedicated mapping rules to ensure structural compatibility with the HL7 FHIR R4 standard. *(Specific extraction mechanisms and integration strategies are detailed in Section 4.1).*
- **Storage & Processing Layer (VNS Data Platform)** The core data engine, strictly enforcing a Medallion Lakehouse architecture to guarantee data lineage, compliance, and analytical performance:
  - **Bronze Layer (Raw Data Store):** The immutable landing zone for raw, unmasked data straight from the Ingestion Layer.
  - **PII Masking & Standardization Engine:** An asynchronous processing engine that dynamically maps local hospital codes to international clinical standards (LOINC, SNOMED CT, ICD-10) and executes strict Personally Identifiable Information (PII) masking.
  - **Silver Layer (FHIR Data Store):** The system of record. It stores the cleansed, masked, and fully compliant HL7 FHIR R4 clinical records.
  - **Gold Layer (Aggregated & Trending):** Utilizing a "Data Trigger" mechanism activated by new Silver data, dedicated compute nodes calculate time-series health trends. The pre-aggregated output lands in the Gold Store, establishing a secure **Patient Tenant Context** (encapsulating masked PII, time-series EMR, and health trends) optimized for AI consumption. *(Deep dive into conversion rules and the dual-storage strategy in Sections 4.3 and 4.4).*
- **Application Layer (Serving & Consumer)** The serving boundary where the Smart Health Checkup App and AI models interface with the platform. All data retrieval is funneled through an MCP (Model Context Protocol) Server and strictly gated by the **SMART on FHIR** authorization protocol. This layer introduces complex security orchestration:
  - **Auth & Access Verification:** Validates standard requests before querying masked data from the Silver/Gold stores.
  - **Re-Identification Service (Secure Vault):** If highly specific authorization criteria are met, this isolated service cross-references the Bronze store to securely unmask patient identities for the end-user.
  - **Audit Logging:** Every interaction via the SMART API is immutably logged for compliance. *(Security gating, the secure vault mechanics, and AI orchestration workflows are expanded upon in Sections 6 and 7).*

@Copyright 2025 by Vietnam Silicon.

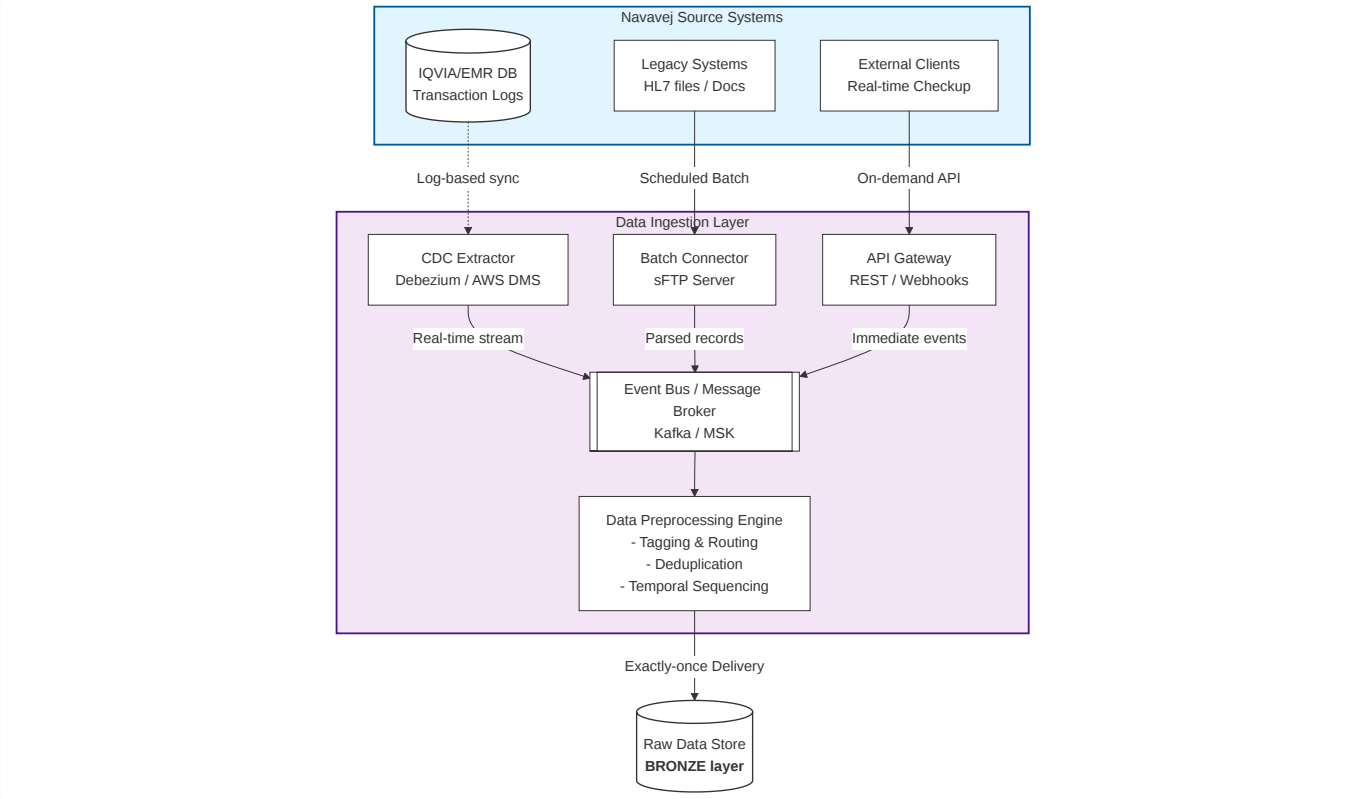
This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

4.2 Data Pipeline at a Glance

The Data Pipeline employs a modern **Medallion Lakehouse Architecture** integrated with Event-Driven processing to transform raw hospital data into AI-ready insights securely:

1. **Ingestion Layer:** Captures raw, unmasked data from Navavej HIS via APIs, Batch, and CDC, landing it directly in the **Bronze Layer**.
2. **Standardization & Masking (Bronze to Silver):** A highly secure pipeline intercepts the raw data, applies strict PII Tokenization/Dynamic Masking to comply with Thailand PDPA and HIPAA, and reformats the clinical elements into HL7 FHIR R4 standards before storing it in the **Silver Layer**.
3. **Curation & Aggregation (Silver to Gold):** Automated ETL/ELT workflows transform the standardized FHIR data into pre-aggregated health metrics, time-series trends, and clinical gaps within the **Gold Layer**, optimized for BI and AI consumption.

4.3 Data Ingestion Layer



› mermaid

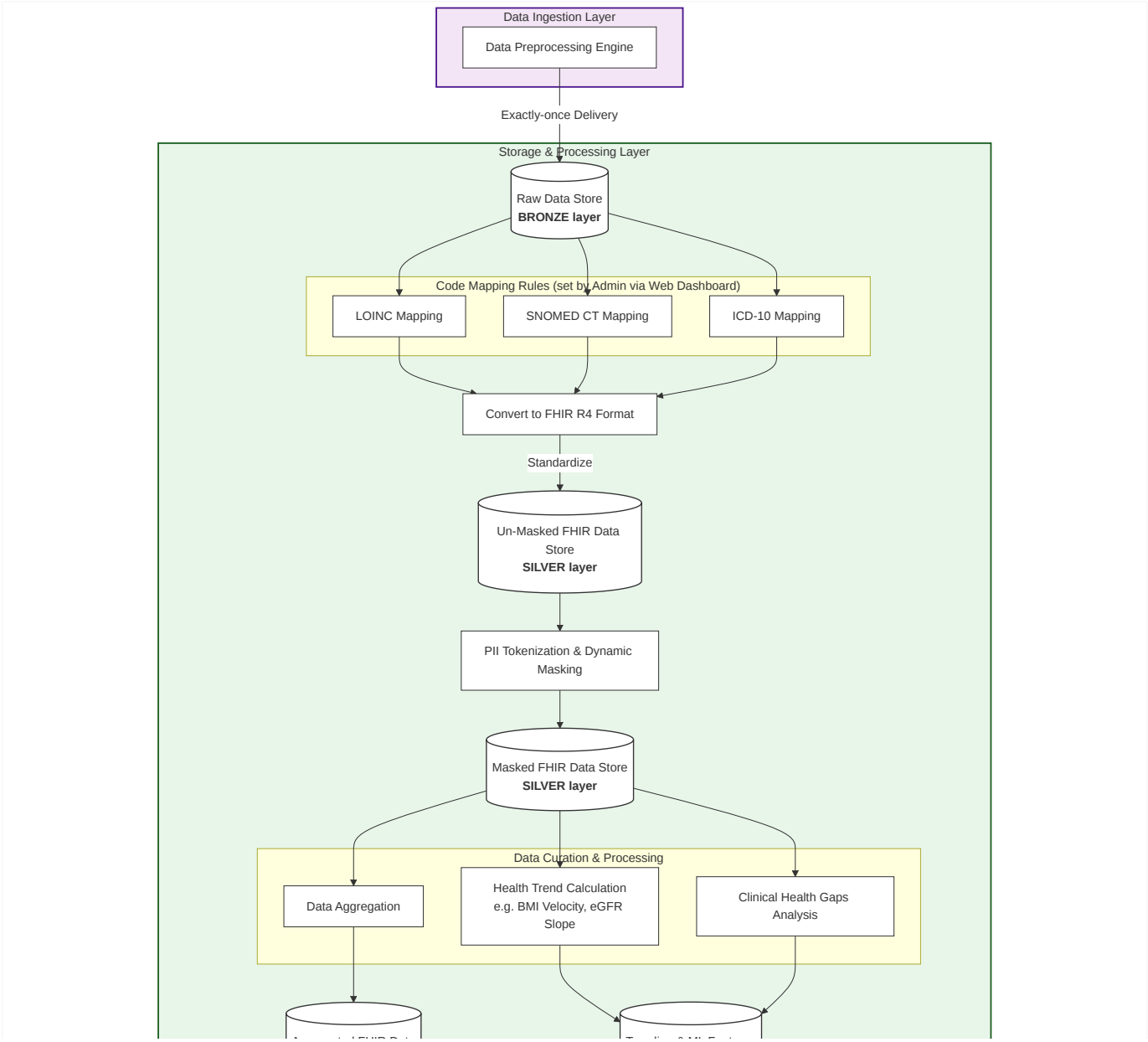
To ensure robust, near real-time data synchronization while minimizing the impact on Navavej's critical hospital operations, we employ a multi-modal ingestion strategy feeding into a unified Event Bus before landing in the Bronze layer.

- **Change Data Capture (CDC):** As our primary real-time ingestion path, we utilize enterprise-grade CDC extractors (e.g., Debezium or AWS DMS). These tools stream continuous, row-level modifications directly from the IQVIA/EMR Operational Store's transaction logs, ensuring low-latency sync without executing heavy queries against the source database.
- **Batch Connectors (sFTP):** For legacy systems, bulk unstructured documents, and scheduled HL7 files, we implement secure sFTP connectors to handle massive data loads reliably during off-peak windows.
- **API Gateways (REST/Webhooks):** To support external clients and real-time smart health checkup requests, on-demand REST APIs and Webhooks capture immediate events and integrate them seamlessly into the platform.
- **Event Bus & Preprocessing Engine:** All ingestion streams converge onto a high-throughput Event Broker (e.g., Apache Kafka or AWS MSK). Before securely landing data into the Bronze Layer, a dedicated Preprocessing Engine applies crucial operations including payload tagging & routing, deduplication, and strict temporal sequencing to guarantee exactly-once delivery and chronological data integrity.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

4.4 Storage & Processing Layer



mermaid

4.4.1 Clinical Code Mapping (LOINC, SNOMED CT, ICD-10)

Standardizing clinical data is a multi-layered process that transforms proprietary Navavej (NVV) local codes into globally recognized terminologies. This ensures that the AI model receives high-fidelity, interoperable data for diagnostic support and health trend analysis.

A. Mapping Architecture: NVV Local Codes to FHIR R4 Standards

The mapping process targets three primary clinical domains: Observations (LOINC), Procedures/Findings (SNOMED CT), and Diagnoses (ICD-10).

Disclaimer: the following table is based solely on `Navavej-Data-Sharing-Package-Master.csv` for hypothetical mapping purposes only.

NVV Local Code	Thai Description (HIS)	FHIR Resource	Standard Coding System	Standard Code	Standard Display Name
NVV-PK-0001	Sleep Test	Observation	LOINC	80351-0	Sleep study
NVV-PK-0004	Chest X-ray	DiagnosticReport	LOINC	36643-5	Chest X-ray 2 views

©Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

NVV-PK-0016	MRI Headache Screening	Condition	SNOMED CT	25064002	Headache
NVV-PK-0017	MRI Dementia Screening	Condition	ICD-10	G30.9	Alzheimer's disease, unspecified
NVV-PK-0018	MRI Stroke Screening	Condition	ICD-10	I63.9	Cerebral infarction, unspecified
NVV-PK-0022	ARI - Acute Respiratory Infection	Condition	ICD-10	J22.9	Acute lower respiratory infection, unspecified
NVV-PK-0025	Influenza Vaccine (Flu)	Immunization	CVX	141	Influenza, seasonal, injectable
NVV-PK-0031	Dental Scaling	Procedure	SNOMED CT	110464002	Dental scaling and polishing
NVV-PK-0032	Colonoscopy	Procedure	SNOMED CT	73761001	Colonoscopy
NVV-PK-0035	Cervical Cancer Screening	Procedure	SNOMED CT	171121004	Screening for malignant neoplasm of cervix
NVV-PK-0043	Bilirubin - Total	Observation	LOINC	1975-2	Bilirubin.total [Mass/volume] in Serum or Plasma
NVV-PK-0072	HBs Ab (Anti HBs)	Observation	LOINC	22322-2	Hepatitis B virus surface Ab [Presence] in Serum or Plasma
NVV-PK-0081	Echocardiogram	Observation	LOINC	18051-3	Echocardiogram

B. Technical Implementation: HAPI FHIR Terminology Service

We leverage the **HAPI FHIR Terminology Service** architecture for high-performance, versioned mapping. The implementation uses specialized handlers inspired by standard FHIR loaders to process NVV-specific CSV/JSON masters.

- **Dynamic ConceptMap Loading**: Local masters from `Navavej-Data-Sharing-Package-Master.csv` are converted into FHIR `ConceptMap` resources.
- **Handler Logic**:
  - **SNOMED CT Handler**: Implements relationship logic (ISA) and concept status tracking (Active/Inactive) as seen in `SctHandlerConcept.java`.
  - **LOINC Handler**: Processes complex lab properties (CodingProperties) and linguistic variants to ensure Thai common names are mapped correctly to LOINC parts.
  - **ICD-10 Loader**: Utilizes `ClaML` (Classification Markup Language) or CSV equivalents to build hierarchical diagnosis trees in the Silver layer.

C. Standard Data Profiling (Synthea-Standardized)

To validate mapping accuracy, the pipeline benchmarks against standardized FHIR datasets (e.g., Synthea).

- **Observation Profiling**: Mapping `pH of Urine` to LOINC `5803-2` and `BMI` to LOINC `59576-4`.
- **Condition Profiling**: Standardizing "Sepsis" to SNOMED CT `91302008`.
- **Encounter Profiling**: Mapping "Urgent care clinic" to SNOMED CT `185347001`.

This mapping ensures that even as HIS codes evolve, the AI layer interacts with an immutable, standardized Gold Layer.

4.4.2 Invalid & Missing Data Handling (HIS Integration)

To maintain high data quality and system reliability, the pipeline implements automated handling for common data issues found in the Navavej HIS:

Data Issue	Handling Strategy	Technical Implementation
Missing `activeto`	**Persistent Defaulting**	If `activeto` is NULL (e.g., `NVV-PK-0006`), the pipeline defaults to `9999-12-31` to represent an indefinitely active package.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Thai Characters in Codes/Names	**UTF-8 Normalization**	Names like `TLI_โปรแกรมตรวจสุขภาพ` are stored in `Observation.code.text` or `Patient.name` using UTF-8. The `code` remains the alphanumeric NVV ID.
Invalid Date Formats	**ISO 8601 Casting**	HIS timestamps (e.g., `2026-03-16 10:50:15.000 +0700`) are cast into standard FHIR UTC format for the Silver layer.
Missing PII (Thai National ID)	**Fallback Identifier**	If Thai National ID is missing, the system uses the Hospital **Medical Record Number (MRN)** as the primary FHIR identifier.
Duplicate Records	**Idempotent UPSERT**	Log-based CDC ensures exactly-once processing. Incoming records with existing `id` or `code` perform a versioned update (FHIR History).
Out-of-Order Events	**Temporal Sequencing**	Using `effectiveTime` from CDC logs to ensure clinical records are processed in the correct chronological order regardless of arrival time.

4.4.3 Health Trend Pre-Computing (Velocity, Gaps, eGFR slope)

To optimize AI performance and ensure reliable medical accuracy, raw standardized data from the Masked Silver layer undergoes heavy transformation in the Processing engine before landing in the Gold Layer. This includes basic Data Aggregation, Health Trend Calculation (e.g., BMI velocity, eGFR slope), and Clinical Health Gaps Analysis. Rather than relying on the AI to deduce complex math on the fly, these generated features are stored as ready-to-query assets.

A. Data Aggregation (Metrics & Summaries)

Basic patient metrics and summaries are aggregated to present holistic health snapshots.

- **Implementation**: Rolls up longitudinal lab results, vital averages over time, and active condition lists. These aggregate views form the foundation of the `Aggregated FHIR Data` Gold store, directly powering BI dashboards and immediate patient history retrieval for the MCP server.

B. Health Trend Calculation (Velocity & Slopes)

eGFR Slope (Renal Function Trend):
The eGFR slope quantifies the annual change in kidney function, serving as a critical surrogate endpoint for chronic kidney disease (CKD) progression.

- **Calculation Methodology**:
  - **Algorithm**: e.g. based on longitudinal `Observation` resources (LOINC `33914-3`, `98979-8`).
  - **Minimum Data**: Requires at least 2 measurements over a minimum 90-day period to establish a baseline and trend.
- **Clinical Significance**: e.g. A slope decline > X mL/min/1.73m²/year indicates rapid progression, triggering an automated clinical alert.

C. BMI Velocity (Growth & Metabolic Trend):
BMI velocity tracks the rate of change in Body Mass Index, essential for identifying early metabolic risks or pediatric growth anomalies.

- **Calculation Methodology**:
  - **Data Source**: Calculated from height (LOINC `8302-2`) and weight (LOINC `29463-7`) observations.
  - **Algorithm**: Rolling 12-month window average.
  - **Normalization**: Adjusted for age and sex using WHO/CDC growth charts for pediatric patients.
- **Clinical Significance**: Sudden spikes in BMI velocity are flagged as early indicators for metabolic syndrome or obesity-related risks.

D. Clinical Health Gaps Analysis

Health gaps identify discrepancies between a patient's current health status and established clinical guidelines.

- **Calculation Methodology**:
  - **Algorithm**: Boolean logic matching `Condition` and `Procedure` history against `CarePlan` protocols.
  - **FHIR Implementation**: Leveraging the `$care-gaps` operation (HAPI FHIR / Clinical Reasoning module) to detect open gaps (e.g., missing HbA1c screening for diabetic patients).
- **Gap Types**:

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

- **Screening Gaps**: Missing age-appropriate screenings (e.g., Colonoscopy, Mammography).
- **Medication Gaps**: Non-adherence to chronic disease medication (e.g., Statins for CVD).

E. Technical Implementation

We construct an automated ETL/ELT pipeline leveraging **Apache Spark** and **dbt** (using `fhir-dbt-analytics` macros).

- **Event-Driven**: Micro-batch recalculations of aggregations and trends are initiated by updates in the Masked Silver layer.
- **Persistence**: Results from Trends and Gaps analysis are structured as engineered features and stored in the **Trending & ML Features Gold layer**, optimized for millisecond-latency retrieval by the Smart Health AI Recommendation Engine.

4.5 Dual-Storage Strategy

4.5.1 Core FHIR Database (Immutable Clinical Records)

The Core FHIR Databases represent our **Silver Layer**, split into two distinct physical stores to ensure absolute privacy:

- **Un-Masked FHIR Store:** Temporarily holds standardized FHIR R4 resources containing PII in a highly restricted secure enclave. Access is strictly limited to the Masking Engine and authorized break-glass services.
- **Masked FHIR Store (System of Record):** The primary system of record for the platform. All Personally Identifiable Information (PII) is removed or tokenized via format-preserving encryption prior to insertion here.
- **Design Pattern:** Data in both stores is strictly modeled as HL7 FHIR R4 JSON resources. The Masked store is treated as an append-only, immutable ledger ensuring exact historical clinical states, full auditability, and uncompromising adherence to PDPA regulations.

4.5.2 Pre-Aggregated Lakehouse (Trending & ML Features)

The Lakehouse represents our **Gold Layer**, optimized exclusively for high-speed analytical reads, complex longitudinal queries, and AI feature serving.

- **Design Pattern:** Optimized for heavy read-access, time-series querying, and columnar analytics rather than transactional clinical updates.
- **Purpose:** It holds the pre-aggregated metrics, statistical gaps, and engineered features (like health velocity and moving averages). This allows the application layer, BI Dashboards, and AI models to retrieve rich patient context with millisecond latency without querying the Core FHIR Database.

5. Data Platform Tech Stack

Our solution is flexible and can be deployed utilizing either AWS Managed Services or an Open-Source Hybrid approach:

AWS Cloud Strategy:

- **Ingestion & CDC:** AWS DMS (Database Migration Service) or Managed Kafka (MSK).
- **Data Storage (Bronze/Silver/Gold):** Amazon S3 (Bronze), AWS HealthLake (Native FHIR Silver Layer - separated into isolated Un-Masked and Masked instances), and Amazon Redshift / Athena (Gold/Trending Store).
- **Processing & PII Masking:** AWS Lambda / AWS Macie for dynamic masking and PII tokenization, AWS Glue / EMR (Apache Spark) for ETL and trend pre-computation.
- **Serving & Security:** Amazon API Gateway, Amazon Cognito (OAuth 2.0 / RBAC), AWS KMS for encryption.

Open Source / Hybrid Strategy:

- **Ingestion & CDC:** Debezium and Apache Kafka.
- **Data Storage (Bronze/Silver/Gold):** MinIO (S3-compatible Bronze), HAPI FHIR backed by PostgreSQL (Silver Layer - with logically separated Un-Masked and Masked schemas), and ClickHouse (High-performance Gold/Trending Store).
- **Processing & Orchestration:** Apache Airflow / Temporal for orchestration, Apache Flink / Baffle for real-time masking, and dbt for metric aggregations.
- **Serving & Security:** Kong / KrakenD API Gateway, Keycloak for Identity/OAuth, and HashiCorp Vault for secure vaults and re-identification.

6. Security, Privacy & Data Governance

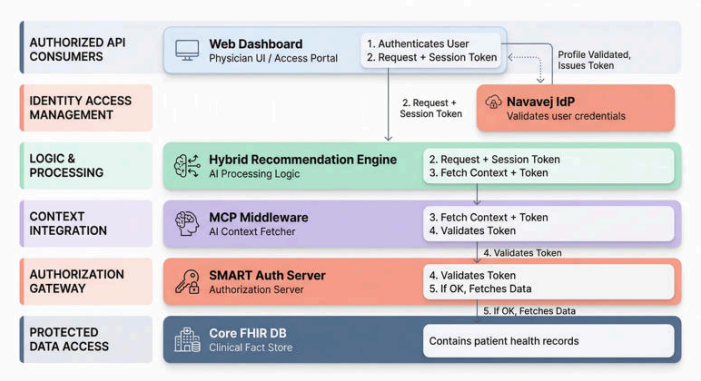
6.1 SMART on FHIR Authorization (Access Control Gatekeeper)

To guarantee absolute compliance with PDPA and healthcare privacy standards, the system implements the **SMART on FHIR** framework as the definitive security gatekeeper for all clinical data. This ensures that no system component—whether it is a doctor's dashboard or the AI Model Context Protocol (MCP) server—can access the FHIR Database without explicitly granted, resource-level permissions.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

6.1.1 High-Level Architecture & Identity Integration



Authentication & Authorization flow with Navavej Identity Provider & SMART on FHIR

- **OAuth 2.0 & Identity Provider (IdP) Integration:** The Authorization Server integrates seamlessly with Navavej Hospital's existing Identity Provider (e.g., Active Directory, Keycloak, or Okta) using standard OAuth 2.0 and OpenID Connect (OIDC). For this platform to operate effectively, the hospital's chosen IdP must satisfy two strict SMART-compliance constraints:
  a. **Define Custom Scopes:** The IdP must natively support scopes containing special characters required by FHIR routing logic (e.g., `patient/*.read` or `launch/patient`).
  b. **Inject Custom Claims:** The IdP must allow the dynamic injection of custom data claims (such as passing a specific patient ID or the `fhirUser` URL) directly into the JWT Access Token before it reaches the FHIR server.
- **Secure Token Relay (On-Behalf-Of Flow):** The architecture utilizes a highly secure, consistent token relay to guarantee that AI inferences always respect human access rights across all microservices:
  a. **Authentication:** A doctor accesses the Web Dashboard and authenticates with the IdP, obtaining an active user session containing a scoped SMART on FHIR access token.
  b. **Recommendation Request:** The dashboard passes the request *along with the doctor's specific session token* to the central backend **Hybrid Recommendation Engine**.
  c. **Context Fetching:** The Hybrid Engine delegates the data-gathering workload to the **MCP Middleware**, passing the user token forward.
  d. **SMART Authorization Gate:** Rather than using an over-privileged generic system password, the MCP presents the doctor's exact session token to the SMART Auth Server. This conclusively proves the components are fetching data strictly *on behalf of* the authorized physician.
  e. **Data Delivery & Masking:** Upon validating the token's scopes and signatures, the SMART server permits the MCP to safely extract the required records from the Core FHIR Database. The MCP then scrubs and masks the PII *before* the Hybrid Engine ever passes it to the AI Gateway for external LLM inference.

6.1.2 Technical Benefits & Security Explanations

Adopting the SMART on FHIR standard provides sweeping technical advantages over traditional API security models:

- **Granular "Least Privilege" Scopes**
  - Traditional systems often grant generic "API access" to an entire database. SMART on FHIR utilizes highly granular clinical scopes (e.g., `patient/Observation.read`). This means the MCP Server can be authorized to read a patient's lab results (Observations) to generate a recommendation, without ever having the technical permission to read their psychiatric notes or billing information.
- **Secure Launch Context (Seamless UX)**
  - The standard perfectly supports "EHR Launch" sequences. If a physician opens our new Recommendation Dashboard directly from inside the legacy Navavej HIS, SMART on FHIR seamlessly passes the active `patient_id` and security token to our app. The doctor does not need to log in a second time or manually search for the patient again, drastically improving user adoption.
- **Open-Source Reliability**
  - Because SMART on FHIR is a global healthcare IT standard, Navavej is not locked into proprietary security logic. The engineering team will leverage battle-tested open-source libraries (such as those from SMART Health IT) to handle the complex token handshakes, ensuring enterprise-grade security while accelerating the development timeline.

6.2 Data Masking & Pseudo-Anonymization Pipelines

To meet the strict `< 10-second` latency SLA (per TOR §6.3), minimize cloud infrastructure costs, and guarantee PDPA/HIPAA compliance, the system adopts an **Asynchronous Pre-Masking Architecture** strategy. Clinical data is scrubbed of all Personally Identifiable Information (PII) at ingestion time—well before AI inference—rather than relying on real-time, on-the-fly masking during the critical

©Copyright 2025 by Vietnam Silicon

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

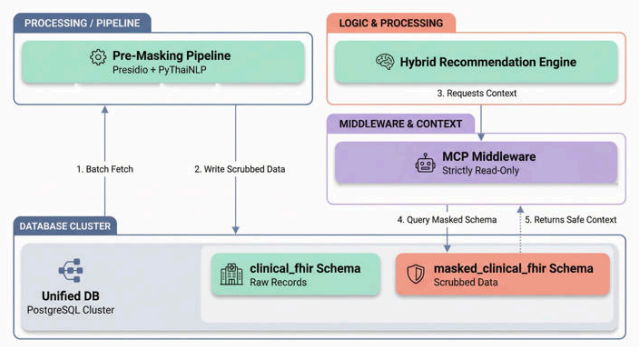
request path. This architectural decision is driven by the computational cost of Thai-language NLP processing, the fail-safe behavior it guarantees, and the auditability requirements of PDPA.

> **Clarification on MCP's Role:** In this architecture, the MCP Middleware operates as a **read-only data accessor** that queries the pre-masked schema exclusively. It does not perform real-time PII scrubbing itself. All NLP-based masking is completed asynchronously by the Data Masking Pipeline before any AI inference request occurs.

6.2.1 Unified Compute, Partitioned Storage Architecture

To reduce cloud infrastructure costs without sacrificing security, the architecture avoids deploying two separate physical database servers. Instead, it utilizes a single, unified database cluster (e.g., PostgreSQL) divided by strict logical schemas with independent access controls.

The Data Integration Pipeline standardizes raw medical data into the protected `clinical_fhir` schema. Immediately after ingestion, an asynchronous background job runs the data through the NLP-based Pre-Masking Pipeline, writing the scrubbed results into an isolated `masked_clinical_fhir` schema on the same server. When the MCP tool (e.g., `getMaskedData?patient_id=123`) is invoked during AI inference, it fetches context exclusively from this pre-scrubbed schema - never from the raw `clinical_fhir` schema.



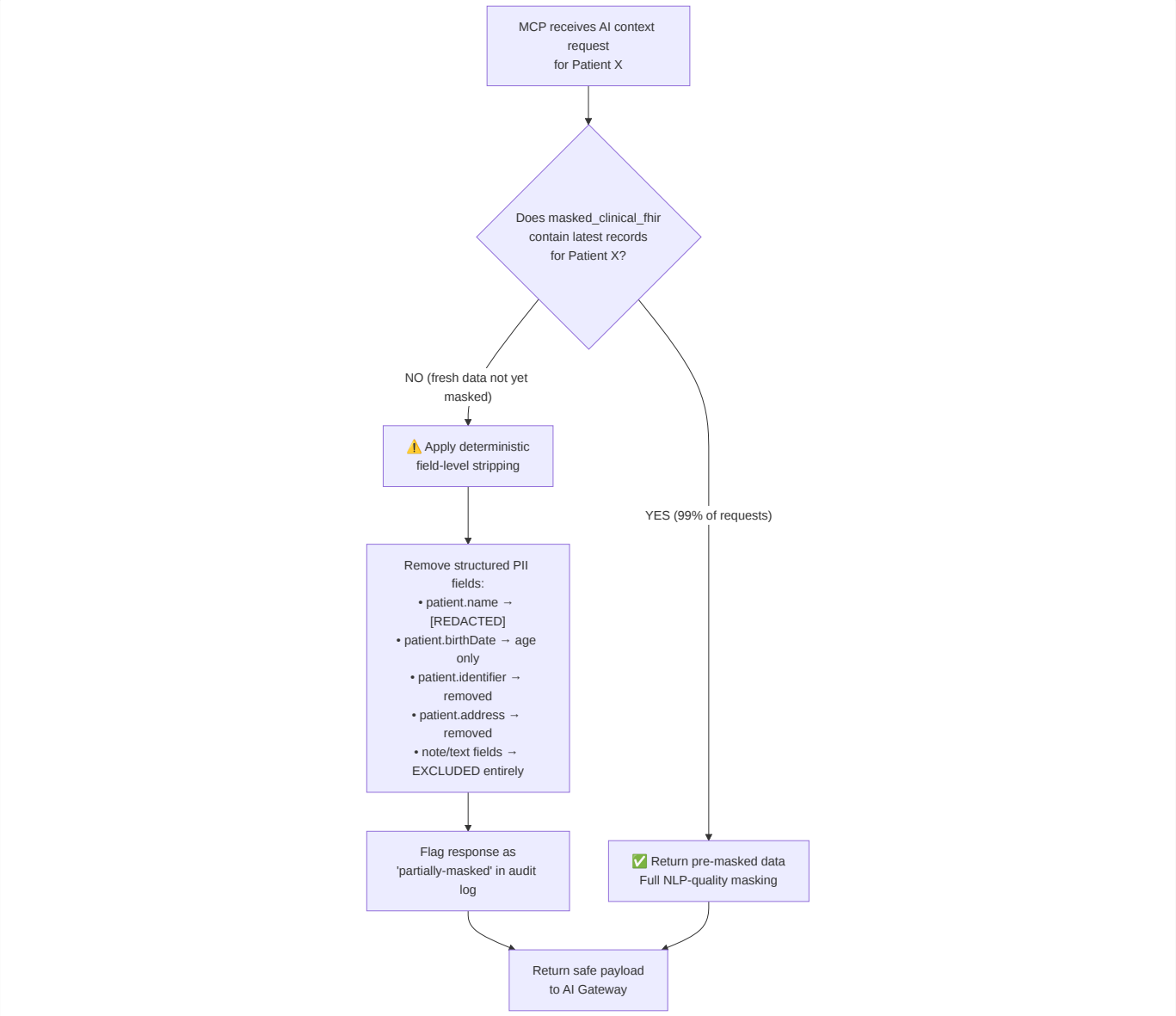
- **Microsoft Presidio & PyThaiNLP Integration:** Because Navavej Hospital serves Thai patients, standard English-focused NLP masking tools are fundamentally insufficient. This pipeline explicitly implements **Microsoft Presidio** (the enterprise-grade OSS standard for PII redaction) customized with **PyThaiNLP**. This guarantees the accurate detection, tokenization, and redaction of Thai names, residential addresses, and National ID patterns hidden within free-text medical notes.

6.2.2 Freshness-Gap Handling (Hybrid Safety Net)

Because the Pre-Masking Pipeline operates asynchronously, there is an inherent **freshness gap**—a brief window (typically seconds to a few minutes) between when new raw data arrives via CDC sync and when its masked counterpart becomes available in the `masked_clinical_fhir` schema. To handle this edge case safely, the system applies a **two-tier masking strategy**:

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon



› mermaid

Tier	Trigger Condition	Masking Method	Coverage
Primary (Tier 1)	Masked copy exists and is up-to-date	Full Presidio + PyThaiNLP NLP pipeline	Complete — structured fields + free-text
Fallback (Tier 2)	New data arrived but masked copy not yet available	Deterministic field-level stripping (no NLP)	Structured fields only — free-text fields are excluded entirely rather than partially masked

- **Why exclude free-text in the fallback?** Free-text clinical notes (physician memos, nursing notes) are the highest-risk PII source and require full NLP processing for safe redaction. Rather than attempting fast but unreliable masking under time pressure, the fallback tier omits free-text entirely, accepting a minor reduction in AI context quality in exchange for guaranteed PII safety. The AI Gateway logs this as a reduced-context inference for audit transparency.
- **Fail-Safe Guarantee:** If both tiers fail (e.g., database connectivity issues), the MCP returns an empty payload. The Hybrid Recommendation Engine then falls back to its deterministic Safety Rules (rule-based only, no AI), ensuring the system never stalls and PII never leaks.

6.2.3 Technical Benefits & Cost Optimization

@Copyright 2025 by Vietnam Silicon.

Adopting pre-masking with a deterministic fallback provides the following advantages over a pure real-time masking approach:

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

- **SLA-Safe Inference Path:** Thai-language NLP masking (word segmentation + NER) on multi-year clinical histories is computationally expensive (estimated 1–5 seconds per patient context). Pre-masking removes this cost entirely from the inference critical path, preserving the full `< 10-second` SLA budget for actual AI reasoning.
- **Infrastructure Cost Efficiency:** Schema-level separation within a single PostgreSQL instance means Navavej pays only for the incremental disk storage of the masked copy (~2× clinical text storage), avoiding the compute cost (vCPU/RAM) of a second dedicated database server or a real-time NLP processing cluster.
- **Role-Based Access "Air Gap":** The two schemas are separated by strict Role-Based Access Control (RBAC). The MCP Middleware's database credentials are granted `SELECT` access exclusively on `masked_clinical_fhir` and explicitly `DENIED` on `clinical_fhir`. Even in the event of a prompt-injection attack reaching the MCP layer, the database enforces the boundary — the MCP cannot technically query raw PII data.
- **Deterministic & Auditable Masking:** Unlike asking an LLM to "mask the patient's name" at inference time (which is non-deterministic and prone to hallucination), the Presidio + PyThaiNLP pipeline is deterministic. The same input always produces the same masked output. Furthermore, the masked data is persisted in the `masked_clinical_fhir` schema, enabling auditors and DPOs to inspect exactly what context the AI received for any historical inference — a key PDPA compliance requirement.
- **Accurate Thai PII Detection:** PyThaiNLP's dictionary-based and statistical word segmentation handles the unique challenges of Thai text (no whitespace, complex compound words) far more reliably than general-purpose LLM instructions. This ensures consistent detection of Thai names (ชื่อ-นามสกุล), National ID patterns (เลขบัตรประชาชน 13 หลัก), and Thai address formats (ที่อยู่) in clinical free-text.
- 5.3 Comprehensive Audit Logging (Tracking the '5 Dimensions' & AI Context)

6.3 Comprehensive Audit Logging

To satisfy PDPA and HIPAA requirements for a clinical AI system, every AI inference operation and physician decision is captured in an immutable, cryptographically protected audit log. The log is written **synchronously before any recommendation response is delivered** — guaranteeing that no inference is ever unrecorded.

6.3.1 5-Dimension Log Structure

Every audit entry is structured around five mandatory dimensions:

Dimension	Fields Captured	Purpose
Who	`user_id`, `user_role`	Identity and role of the healthcare staff who triggered the inference (e.g., physician, nurse, pharmacist)
When	`timestamp` (ISO 8601, UTC)	Precise moment of the inference or clinical decision event
Where	`service_id`, `client_ip`, `request_id`	Originating service and network location of the request, enabling full source traceability
What	`scrubbed_ai_context` (full MCP context sent to the LLM, with PII removed), `ai_response`, `model_id`, `latency_ms`, `safety_conflict_events`	The complete transaction record of what was sent to the AI and what it returned. All patient-identifiable fields are **scrubbed before inference** — no PII reaches the AI or this log in any form
Why	`patient_id`, `action_type`, `request_id`	The clinical purpose that triggered the inference

6.3.2 Immutability: SHA-256 Hash Chaining

Immutability is enforced through cryptographic hash chaining. Each log entry computes:

```
current_hash = SHA-256(previous_entry_hash + current_payload)
```

The chain is anchored by a known genesis hash. Any post-write modification — including deletion, reordering, or content edits — produces a detectable hash mismatch across all subsequent entries. This provides tamper-evidence without requiring an external blockchain or notarization service.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

6.3.3 Physician Decision Logging

Approval and Rejection decisions by healthcare providers are recorded as a distinct event type. Each decision entry is linked to the original AI recommendation via a `recommendation_reference_hash`, creating a fully traceable audit trail:

```
Clinical Data → AI Inference → Human Review → Final Decision
```

6.3.4 PII Enforcement

Patient-identifiable fields are **scrubbed** from all clinical data **at ingestion time** — not at inference time — by the asynchronous Pre-Masking Pipeline (Presidio + PyThaiNLP), which writes de-identified records into the isolated `masked_clinical_fhir` schema. The MCP Middleware reads exclusively from this pre-scrubbed schema, ensuring the AI inference pipeline only ever operates on de-identified clinical signals. No PII is substituted, tokenised, or rehydrated within the backend at any stage of the AI workflow. Patient identity is resolved solely by the frontend at display time, using a secure SMART on FHIR-scoped lookup against the `clinical_fhir` schema — a path that is architecturally separated from and never intersects with the AI pipeline.

6.3.5 Retention

All audit records must be retained for a minimum of **3 years** per PDPA Section 40. Specify storage infrastructure, cold-tier archival strategy (e.g., object storage lifecycle rules after 6 months), and automated purge/archival mechanism at the 3-year mark.

6.3.6 Access Control

Read access to audit logs is restricted to the **DPO** and **System Admin** roles. Clinical staff (physician, nurse, pharmacist) cannot query or export audit records directly.

Refer to with §6.1.

7. AI & Recommendation Engine Architecture

7.1 High-Level AI Workflow

The AI recommendation pipeline is orchestrated as a four-phase asynchronous loop. To balance inference quality against latency and cost, the pipeline supports two complementary execution paths that converge on the same three-agent chain.

7.1.1 Two-Path Design

Optimization Path (default) — For cases where the standard clinical data set is sufficient, four MCP context tools are fetched in parallel upfront. If the assembled context is deemed complete, the pipeline proceeds directly to the agent chain with no additional tool calls.

ReAct Path (data-gap cases) — When the AI determines that the prefetched context is insufficient for a high-quality recommendation, it issues additional targeted MCP tool calls before proceeding. The number of additional calls is bounded by a configured iteration cap to prevent unbounded latency.

Both paths converge at Agent 1 and follow the same agent chain and post-processing steps from that point forward.

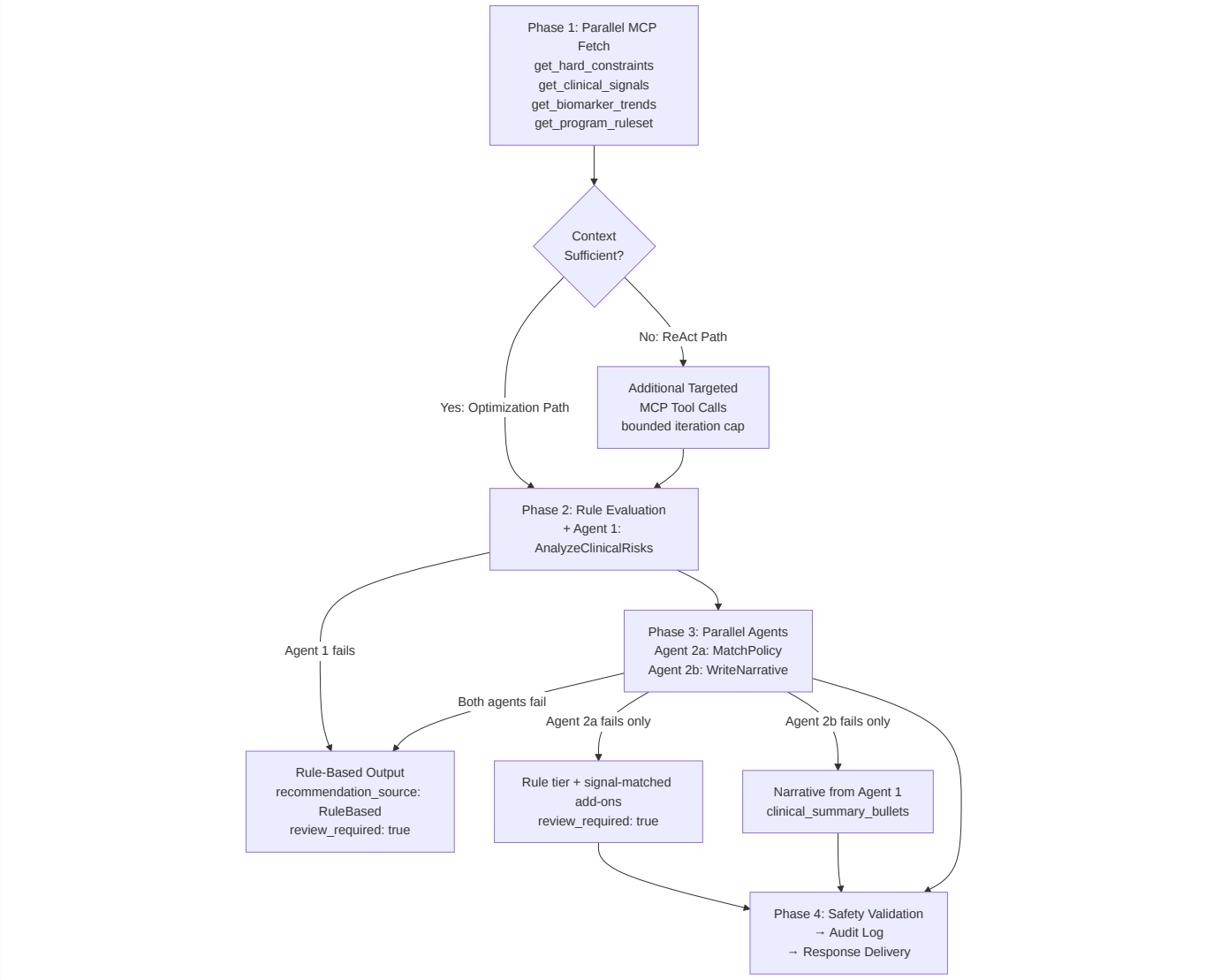
7.1.2 Four-Phase Pipeline

Phase	Steps	Notes
Phase 1 — Context Fetch	4 MCP tools called in parallel (`get_hard_constraints`, `get_clinical_signals`, `get_biomarker_trends`, `get_program_ruleset`)	No ordering dependency between tools; all results assembled before proceeding
Phase 2 — Rule Evaluation + Agent 1	Deterministic rule tier computed from active signals; Agent 1 (`AnalyzeClinicalRisks`) produces a structured clinical risk profile	Agent 1 failure triggers immediate full rule-based fallback
Phase 3 — Agent 2a + Agent 2b (parallel)	Agent 2a (`MatchPolicy`) selects program tier and add-ons; Agent 2b (`WriteNarrative`) produces patient-facing communication script; both run concurrently	Either agent can fail independently; per-field fallback applied
Phase 4 — Safety Validation + Audit + Response	`SafetyRuleValidator` filters gender/age/contraindication-ineligible add-ons; audit log written synchronously; response delivered to caller	Frontend resolves patient identity from `patient_id` at display time

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

7.1.3 Workflow Diagram



AI Workflow Diagram

7.1.4 Graceful Degradation

The pipeline is designed to always produce a recommendation. Failures are handled at the coarsest applicable boundary:

- **Agent 1 failure** → full rule-based output; AI chain bypassed entirely
- **Agent 2a (`MatchPolicy`) failure only** → rule-based tier applied; signal-matched add-ons substituted; `review_required: true`
- **Agent 2b (`WriteNarrative`) failure only** → narrative constructed from Agent 1 clinical summary bullets
- **Both Agent 2a and 2b failure** → full rule-based output

In degraded states where the AI tier is unavailable (Agent 1 failure, Agent 2a failure, or both Agent 2a and 2b failure), `recommendation_source` is set to `"RuleBased"` and `review_required` is set to `true`, signalling to the clinical UI that human review is required before acting on the recommendation. When only Agent 2b fails, the AI-selected tier and add-ons are retained and `recommendation_source` remains `"AI"` — only the patient-facing narrative is degraded.

7.2 Model Context Protocol (MCP) Middleware (Secure Context Retrieval)

The Model Context Protocol (MCP) middleware layer sits between the clinical data platform and the AI inference pipeline. It exposes a standardised set of read-only tool interfaces that the AI agents use to retrieve structured patient context on demand. This section defines the **MCP → AI interface**: the tools, the clinical data they expose, and how each is consumed.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

> **Data Platform Specifications:**
>
> - **Datastore Technology:** Tools retrieving raw clinical records (`get_hard_constraints` , `get_patient_event_timeline` , `get_program_ruleset`) query the **Silver Layer** (Masked FHIR Store via PostgreSQL/AWS HealthLake). Analytical tools (`get_clinical_signals` , `get_biomarker_trends`) query the **Gold Layer** (Lakehouse via ClickHouse/Amazon Redshift) for pre-computed trends and metrics.
> - **Ingestion Pipeline:** Event-driven architecture using CDC (Debezium/AWS DMS) from IQVIA/EMR, feeding through an event bus (Kafka/MSK) into Bronze storage, followed by standardization, PII masking (Silver), and metric aggregation (Gold).
> - **Refresh Cadence:** Near real-time (<100ms latency) for continuous CDC streams; micro-batch updates for complex Gold layer trends. Legacy/bulk imports via nightly batch (4-hour window).
> - **Availability SLAs:** 99.9% system uptime via Multi-AZ deployment. Sub-500ms latency for FHIR queries (Silver) and sub-200ms latency for analytics queries (Gold).

7.2.1 Tool Interface

The MCP server exposes five read-only tools, each scoped to a single patient identified by `patient_token` . Four tools feed the AI inference pipeline; one feeds the clinical UI directly and is never sent to an LLM.

Tool	Clinical Data Exposed	Consumer
`get_hard_constraints`	Patient demographics, active safety flags, eligibility exclusions, tier ceiling	AI inference — all 3 agents
`get_clinical_signals`	Fired signals, preliminary rule tier, overall risk level, missing signals, risk/comorbidity/screening flags	AI inference — all 3 agents
`get_biomarker_trends`	BMI, blood pressure, eGFR, lab values, ASCVD 10-year risk	AI inference — all 3 agents
`get_program_ruleset`	Tier selection rules, add-on item library, package resolution map	AI inference + deterministic rule evaluation
`get_patient_event_timeline`	Diagnoses, hospitalizations, screening history, medication changes	UI display only — **not sent to AI**

7.2.2 Design Constraints

- All tools are **read-only** — no tool writes to or modifies any clinical data store.
- Tool calls are **single-patient-scoped** — each call retrieves data for exactly one patient.
- The four AI-facing tools are called **in parallel** during Phase 1 of the workflow (see §7.1), with no ordering dependency between them.
- `get_patient_event_timeline` is called separately by the UI layer and is excluded from the AI context assembly.

7.3 Hybrid Recommendation Logic (Rule-based Safety + AI Inference)

The recommendation engine uses a three-layer decision architecture that combines deterministic clinical rules with AI-driven contextual reasoning. Each layer has a fixed responsibility; the ordering ensures that AI judgement is always grounded in rule-computed clinical context and then checked by a final deterministic safety pass before any recommendation reaches a physician.

7.3.1 Three-Layer Architecture

Layer 1 — Deterministic Rule Tier (pre-AI) The `RuleEvaluator` processes the patient's active clinical signals and maps them to a recommendation tier using an admin-configurable rule set. Rules are evaluated in priority order; the first matching rule determines the tier. This tier serves as the anchor value passed to Layer 2.

Layer 2 — AI-Driven Tier and Add-On Selection (Agent 2a) Agent 2a (`MatchPolicy`) receives both the assembled clinical context and the rule-computed preliminary tier. It selects the final tier and recommends specific add-on items. If the AI selects a tier that differs from the rule-computed

@Copyright 2025 by Vietnam Silicon

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

anchor, it must provide a written `tier_deviation_justification` — this field is mandatory and non-nullable when a deviation occurs, ensuring that every AI override is explicitly reasoned and auditable.

Layer 3 — Post-AI Safety Filter (deterministic) `SafetyRuleValidator` applies deterministic eligibility checks against the AI-recommended add-ons. Any add-on that fails a gender, age, or contraindication constraint is silently removed from the final list. Each removal is logged as a `safety_conflict_event` in the audit record. This layer cannot be overridden by AI output.Tier Selection Rules

Rules are evaluated in priority order. The first matching rule is applied.

Priority	Rule ID	Tier	Match Logic	Trigger Condition
1	`PREMIUM_SEVERE_SINGLE`	Premium	Any single severe indicator	eGFR annual decline > 5 mL/min/1.73m², ASCVD 10-yr risk ≥ 20%, hypertensive crisis (SBP > 180 or DBP > 120), confirmed DM with active organ complication screening gap, or significant liver enzyme elevation (ALT > 3× ULN) with NAFLD/alcohol/high-TG indicator
2	`PREMIUM_MULTI_SIGNAL`	Premium	Cumulative signal burden	≥ 5 of 12 moderate-to-high-risk signals simultaneously active (e.g. prediabetes, HTN, dyslipidaemia, BMI obese by Asian threshold, intermediate ASCVD, stage 2 HTN, family history of premature CAD, at-risk alcohol use, fibroscan indication, lung CT eligibility, CKD, hyperuricaemia)
3	`ADVANCED_METABOLIC_OR_CVD`	Advanced	Single metabolic/CVD signal or moderate co-occurring burden	Prediabetes or diabetes confirmation pending, intermediate ASCVD (7.5–19.9% 10-yr risk), Stage 1–2 HTN with comorbidity, LDCT eligibility, CKD (any stage), BMI ≥ 27.5 kg/m² (Asian threshold), or ≥ 2 of 14 moderate-risk indicators
4	`STANDARD_DEFAULT`	Standard	Default fallback	No elevated-risk signals present; routine annual preventive screening

7.3.2 Add-On Priority Levels

Each recommended add-on item carries a priority designation that drives the clinical UI's display order and urgency signalling.

Priority	Clinical Indication	Examples
Red	Urgent finding — high-severity condition, active symptom, or mandatory screening gap severely overdue	Lung CT (LDCT-eligible), colonoscopy with haematochezia or first-degree CRC history, Exercise Stress Test (ASCVD ≥ 20%), FibroTouch (ALT > 3× ULN), Diabetes Screening Pro (confirmation pending)
Orange	Moderately important — standard screening overdue within accepted window or elevated risk requiring near-term attention	Cervical cancer screening (standard overdue), DEXA bone density (age ≥ 65), Carotid ultrasound (ASCVD 15–19.9%), Thyroid Pro 2 (thyroid dysfunction likely)
Yellow	Enhanced monitoring recommended — borderline or developing risk, or patient-specific surveillance	Exercise Stress Test (intermediate ASCVD), Coronary Artery Calcium scoring, Echocardiogram (DM or HTN comorbidity), 48-Hour Cardiac Patch Monitor (palpitations/syncope), Fibroscan (liver disease risk), Retinal Examination (diabetes)
Green	Shared decision-making / discretionary — USPSTF Grade C recommendation or patient-preference-dependent	PSA (age 55–69, shared decision-making per USPSTF Grade C), Spine MRI (occupational or chronic pain, discretionary)

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

7.3.3 Safety Validation Checks

After AI-driven add-on selection, `SafetyRuleValidator` applies four deterministic checks against each recommended item:

1. **Item validity** — item ID must exist in the current add-on library
2. **Gender constraint** — add-ons with a gender restriction are removed if the patient does not match
3. **Age bounds** — add-ons with a minimum or maximum age requirement are removed if the patient is outside the eligible range
4. **Contraindication signals** — add-ons that define contraindication signals are removed if any of those signals are active for the patient

Removed items do not appear in the final recommendation. Each removal is recorded as a `safety_conflict_event` with the item ID and violation reason, captured in the `What` dimension of the audit log.

7.3.4 Fallback Chain

If AI components are unavailable, the recommendation engine degrades in steps, always producing a usable output:

Failure Scenario	Output
Agent 1 (`AnalyzeClinicalRisks`) fails	Full rule-based recommendation; AI pipeline bypassed
Agent 2a (`MatchPolicy`) fails, Agent 2b succeeds	Rule-based tier applied; signal-matched add-ons used; `review_required: true`
Agent 2b (`WriteNarrative`) fails, Agent 2a succeeds	Narrative constructed from Agent 1 clinical summary bullets
Both Agent 2a and 2b fail	Full rule-based recommendation; `review_required: true`

In degraded states where the AI tier is unavailable (Agent 1 failure, Agent 2a failure, or both Agent 2a and 2b failure), `recommendation_source` is set to `"RuleBased"` and `review_required` is set to `true`. When only Agent 2b fails, the AI-selected tier and add-ons are retained and `recommendation_source` remains `"AI"` — only the patient-facing narrative is degraded.

7.4 Central AI Gateway

The Central AI Gateway is a dedicated routing and abstraction layer that sits between the clinical application and all external LLM providers. Its purpose is to ensure that the clinical agent pipeline has no direct dependency on any specific AI provider — model selection, fallback behaviour, timeout enforcement, and credential management are all handled at the gateway layer, invisible to the application code above it.

7.4.1 Responsibilities

Provider abstraction. Clinical agents issue inference requests through a single, stable interface. The underlying model — whether from a cloud AI lab, an open-source deployment, or an on-premise server — is resolved at runtime by the gateway. No provider-specific client code exists in the application layer.

Automatic fallback chain. The gateway maintains an ordered list of model candidates. If the configured primary model is unavailable, times out, or returns an unrecoverable error, the gateway automatically retries with the next candidate in the chain. This process is transparent to the clinical agent — the agent receives a valid response or a clean failure, never a partially-handled provider error.

Per-call timeout enforcement. Every inference call is subject to a maximum wall-clock timeout, configured at the gateway level. This bounds worst-case latency regardless of provider behaviour and prevents a slow or unresponsive model from blocking the recommendation pipeline.

Credential isolation. Provider API keys are managed via environment configuration and are never referenced in application code. Rotating credentials or changing providers requires no code change.

Structured output contract. The gateway enforces that every model response conforms to the typed output schema expected by the calling agent. Raw unstructured text never reaches the clinical logic layer — schema validation failures are handled at the gateway boundary, not scattered through the application.

7.4.2 Relationship to Clinical Agents

All three clinical agents — `AnalyzeClinicalRisks` , `MatchPolicy` , and `WriteNarrative` — route their inference calls through the gateway. They are authored against the typed contract; the gateway resolves which model fulfils it at runtime. This separation allows the active model to be changed without touching any agent definition.

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

7.4.3 Explainable AI (XAI) & Confidence Scoring Generation

Every recommendation produced by the AI pipeline includes a structured explanation and a system-computed confidence score. These are designed to support physician judgement — not replace it — by making the AI's clinical reasoning transparent and quantifiable.

Reasoning Trace

The reasoning trace is assembled from two sources within the AI pipeline:

- **Clinical risk profile bullets** — produced by Agent 1 (`AnalyzeClinicalRisks`): a structured set of identified risk factors, severity classifications, and contraindication flags derived from the patient's clinical signals
- **Policy reasoning** — produced by Agent 2a (`MatchPolicy`): the AI's written justification for the selected tier and add-on items, including a mandatory deviation justification if the AI overrides the rule-computed preliminary tier

These two outputs are combined into the `reasoning_trace` field exposed in the API response, giving the physician a readable account of why a specific recommendation was generated.

Confidence Score

The confidence score is computed deterministically by the system after AI inference — it is not a self-reported probability from the LLM. This design prevents the model from producing artificially high confidence values and ensures the score reflects the actual completeness of the available clinical data.

The formula:

```
confidence_score = 0.4 × signal_coverage + 0.4 × data_completeness + 0.2 × rule_agreement
```

Component	Definition	Weight
`signal_coverage`	Proportion of expected clinical signals for which data was available	40%
`data_completeness`	Proportion of required output fields successfully populated	40%
`rule_agreement`	1.0 if the AI-selected tier matches the deterministic rule tier; 0.0 if it deviates	20%

The score is expressed as a value between 0 and 1, rounded to three decimal places.

Confidence Tiers

Tier	Score Range	Behaviour
High	≥ 0.75	Recommendation delivered without mandatory review flag
Medium	0.50 – 0.74	Recommendation delivered; physician discretion advised
Low	< 0.50	`review_required: true` set; clinical UI flags recommendation for mandatory physician review before acting

> **Note:** Confidence tier thresholds are pending confirmation with clinical stakeholders and may be adjusted based on clinical validation results.

Communication Script

Agent 2a and Agent 2b jointly produce a four-field patient-facing communication script that accompanies the recommendation:

Field	Content
`health_summary`	Plain-language summary of the patient's current health status and key findings
`recommendation_rationale`	Explanation of why this programme tier and these add-ons were selected

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

`addon_highlights`	Concise clinical rationale for each recommended add-on item
`next_steps`	Actionable guidance for the patient on what to expect and how to proceed

All XAI fields — the reasoning trace, confidence score, confidence tier, and communication script — are included in the API response and available to the clinical UI for physician display.

7.4.4 Pluggable LLM Configuration Management

The AI gateway is designed to be fully provider-agnostic. The active model is a **configuration value**, not a code dependency. Switching from one model to another — whether across providers, model sizes, or deployment modes — requires a single configuration update and a service restart. No application code changes, no redeployment, and no schema migration are needed.

Zero Vendor Lock-In

This architecture was a deliberate design choice to protect the project against the rapid pace of change in the AI model landscape. As newer, more capable models become available from different providers, the hospital is free to adopt them immediately. Similarly, if cost, data sovereignty, or regulatory requirements shift — such as a need to move from cloud inference to fully on-premise — the transition can be made at the configuration layer without touching any clinical logic.

Supported Provider Categories

Provider Category	Example Models	Deployment Mode
Cloud AI labs	• GPT-5.x (OpenAI) • Claude 4.x (Anthropic) • Gemini 3.x (Google) • and equivalents	Cloud
Open-source self-hosted	• Llama • Mistral • Qwen • and compatible open-weight models	On-premise or air-gapped
Model aggregator	• OpenRouter • TogetherAI	Cloud

Configuration Model

The active model is selected by setting a single environment variable at deployment time. The fallback chain — the ordered list of candidates the gateway will try if the primary model fails — is also declared in configuration, not in code.

This means the same application build can serve different model configurations across environments: a lightweight open-source model for development and testing, a proprietary cloud model for production, or a fully air-gapped on-premise model for a data-sovereign deployment — all without branching the codebase.

7.5 AI Tech Stack

Component	Technology	Role
Structured LLM Prompting	BAML (BoundaryML)	Type-safe prompt authoring and output schema enforcement for all three clinical agents
AI Orchestration	FastAPI + asyncio	Asynchronous 4-phase workflow coordination; parallel agent execution
Clinical Context Middleware	MCP (FastMCP)	Read-only clinical context tool server; stdio transport
LLM Providers	OpenAI / Anthropic / Ollama / OpenRouter / Self-hosting	Multi-provider support via the AI gateway fallback chain
LLM Client SDK	BAML unified SDK	BAML's unified OpenAI-compatible client for cloud and self-hosted model endpoints

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Configuration	Pydantic + pydantic-settings	Environment-variable-driven configuration, including active model selection
Logging	Loguru	Structured asynchronous logging
Audit Storage	JSONL + SHA-256	Append-only, hash-chained immutable audit trail
Caching	In-memory TTL cache	Risk profile cache keyed on a hash of the scrubbed clinical signal set
ASGI Server	Uvicorn	Production asynchronous HTTP server

8. Frontend & Dashboards (User Experience)

The frontend of the Smart Health Checkup Recommendation Dashboard is designed as a modern, containerized **Single-Page Application (SPA)** built entirely on the **React** ecosystem. This section defines the technology stack, the high-level component architecture mapping every TOR deliverable (§3.3 and §3.4) to a concrete UI module, and the containerization strategy that ensures the frontend can be independently deployed, scaled, and swapped alongside all other microservices.

8.1 Responsive Web Application Architecture

8.1.1 Technology Stack

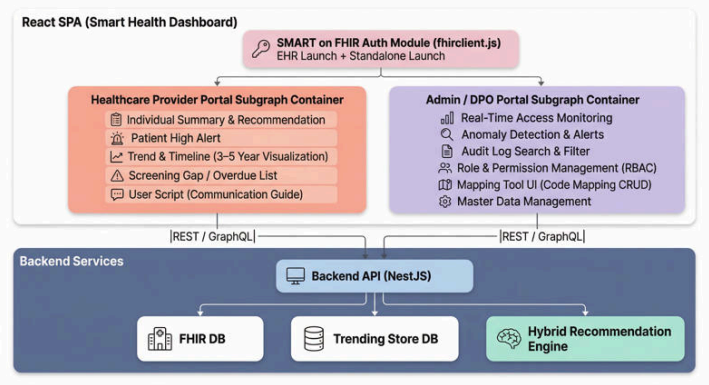
Layer	Technology	Purpose
UI Framework	React 18+	Component-based SPA with concurrent rendering for non-blocking UX (per TOR §6.3)
Language	TypeScript	Type safety across the entire frontend codebase, reducing runtime errors in clinical data rendering
Routing	React Router v6	Client-side role-based routing (Healthcare Provider routes vs. Admin/DPO routes)
State Management	React Query (TanStack Query)	Server-state caching, background re-fetching, and optimistic updates for real-time dashboards
Charting / Visualization	Recharts or Apache ECharts	Interactive trend lines, timeline visualizations, and anomaly detection graphs (TOR §3.3 item 3)
Data Tables	TanStack Table	Sortable, filterable, paginated tables for audit logs, mapping tools, and overdue lists
Form Management	React Hook Form + Zod	Schema-validated forms for the Mapping Tool UI and Master Data Management screens
Authentication	`fhirclient.js` (SMART Health IT)	Official SMART on FHIR client library for OAuth 2.0 launch, token management, and EHR Launch context
Internationalization	`react-i18next`	Thai and English language support for all UI labels, user scripts, and clinical terminology
Styling	CSS Modules or Styled Components	Scoped, component-level styling with design token support
Build & Bundle	Vite	Fast HMR during development, optimized production builds with tree-shaking
Containerization	Docker (multi-stage build)	Lightweight Nginx-based production image (see §7.1.5)

8.1.2 High-Level Component Architecture

The following diagram maps every TOR-required screen (§3.3 items 1–5, §3.4 items 1–6) to a concrete React module within the application. The SPA is split into two role-gated areas: the **Healthcare Provider Portal** and the **Admin / DPO Portal**.

@Copyright 2025 Vietnam Silicon

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon



› mermaid

8.1.3 Component-to-TOR Requirement Mapping

Each React module directly addresses a specific TOR deliverable. The table below provides a precise mapping and highlights the key UI behaviors expected:

Healthcare Provider Portal (TOR §3.3)

TOR Ref	Component	Key UI Behaviors
§3.3 item 1	**Individual Summary & Recommendation**	Displays `Recommended Program`, `Add-on List`, `Reasoning` (Explainable AI narrative), `Priority Level` (color-coded: Red / Yellow / Green), and `Confidence Score` (percentage bar). Uses React Query's `useQuery` to fetch AI recommendations asynchronously, rendering a skeleton/loading state to satisfy the non-blocking UI requirement (TOR §6.3).
§3.3 item 2	**Patient High Alert**	A prominently positioned alert panel showing abnormal values affecting program selection, alarming lab values at alert levels, and incomplete/missing records (Dead Record / Missing Critical Data). Rendered as dismissible alert cards with severity badges.
§3.3 item 3	**Trend & Timeline**	Interactive multi-series line charts (Recharts/ECharts) showing 3–5 years of longitudinal health data (BMI velocity, eGFR slope, lab value trends). A synchronized event timeline below the chart marks significant events (abnormal values, hospital admissions, diagnosis dates). Users can zoom, pan, and hover for detail tooltips.
§3.3 item 4	**Screening Gap / Overdue List**	A prioritized table of examinations that are due or overdue, with `Overdue Score` driving the sort order. Each row displays the screening type, last examination date, days overdue, and a color-coded urgency indicator (Red = urgent, Yellow = approaching). Includes vaccination status tracking (Influenza, Hepatitis B, HPV, Pneumococcal).
§3.3 item 5	**User Script (Communication Guide)**	A collapsible panel providing standardized physician-to-patient communication scripts. Scripts are dynamically generated based on the active recommendation, ensuring explanations remain consistent with hospital standards. Supports copy-to-clipboard and print functionality.

Admin / DPO Portal (TOR §3.4)

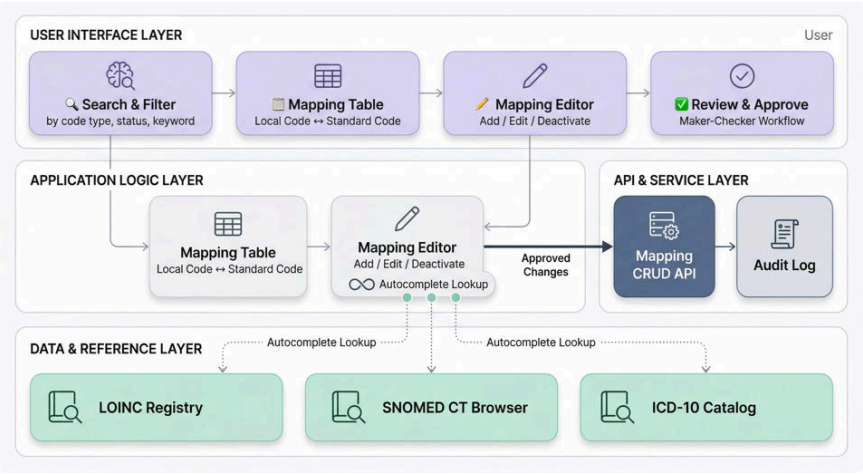
@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

TOR Ref	Component	Key UI Behaviors
§3.4.1	**Real-Time Access Monitoring**	Live dashboard showing active users count, patient record access frequency, and access history feed. Uses WebSocket or Server-Sent Events (SSE) for real-time updates without polling.
§3.4.2	**Anomaly Detection & Alerts**	Displays behavioral anomaly alerts: bulk record access in short periods, accessing unrelated patient data, deviation from a user's baseline behavior pattern. Alert cards link directly to the Audit Log Search for investigation.
§3.4.3	**Audit Log Search & Filter**	Full-featured search interface with filters for: Username, Patient ID, Date/Time range, and Activity type (View / Edit / Export). Powered by TanStack Table with server-side pagination for large audit datasets. Results are exportable for compliance reporting.
§3.4.4	**Role & Permission Management**	RBAC administration screen allowing permission assignment by job position or department, granular separation of View / Edit / Export permissions, and dynamic role adjustment. Changes are audit-logged.
§3.4.5	**Mapping Tool UI**	See §8.1.4 below (dedicated subsection).
§3.4.6	**Master Data Management**	CRUD interface for managing core system reference data: health checkup program lists, alert threshold criteria, priority assignment logic, vaccine lists, standard screening items, and the medical rules applied within the recommendation engine. All changes are version-tracked and require approval workflow before taking effect in production inference.

8.1.4 Mapping Tool UI — Clinical Code Management (TOR §3.4.5)

The TOR explicitly requires a UI-based tool that allows authorized staff to manage clinical code mappings **without requiring developer support**. This is a critical administrative component that bridges the gap between Navavej's local HIS codes and the international standards (LOINC, SNOMED CT, ICD-10) used by the FHIR database and Data processing pipeline.



Mapping tool - The flow and related components

› mermaid

Core Capabilities:

Feature	Description
Multi-Domain Mapping	Supports four mapping categories: Lab codes (→ LOINC), Diagnosis codes (→ SNOMED CT / ICD-10), Procedure codes (→ SNOMED CT), and Health Checkup Package codes (→ internal FHIR Code).
Autocomplete with Standard Code Lookup	When creating or editing a mapping, the editor provides an autocomplete search against the LOINC, SNOMED CT, and ICD-10 reference databases,

@Copyright 2025 by Vietnam Silicon.

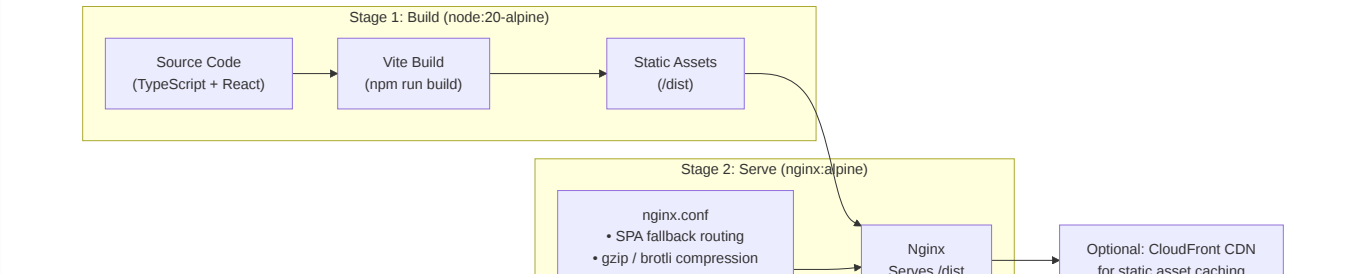
This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

	showing the official code, display name, and description. This prevents incorrect mappings.
Maker-Checker Approval Workflow	To prevent accidental or unauthorized changes to production mappings, all modifications follow a two-step workflow: one authorized user creates/edits the mapping (Maker), and a second authorized user reviews and approves it (Checker) before it takes effect.
Bulk Import / Export	Authorized staff can upload CSV/Excel files for bulk mapping updates (e.g., when onboarding a new lab panel) and export the current mapping table for offline review.
Version History & Rollback	Every mapping change is version-tracked. Administrators can view the full change history of any mapping entry and roll back to a previous version if needed.
Audit Trail Integration	All mapping actions (create, edit, approve, rollback) are recorded in the 5-Dimension Audit Log (Who, When, Where, What, Why) for PDPA compliance.

Access Control: Only users with the `mapping_admin` role (assigned via the Role & Permission Management screen, §3.4.4) can access the Mapping Tool. View-only access is available for clinical staff who need to verify mappings but should not modify them.

8.1.5 Containerization & Deployment (TOR §4.3)

Per TOR §4.3, all services—including the frontend—must run as independent Docker containers to facilitate convenient plug/unplug operations. The React SPA is packaged using a **multi-stage Docker build** to produce a minimal, production-optimized image:



mermaid

Dockerfile Strategy (Multi-Stage Build)

Stage	Base Image	Purpose	Output
Build	`node:20-alpine`	Install dependencies, compile TypeScript, run Vite production build	Static HTML/JS/CSS bundle in `/dist`
Production	`nginx:alpine`	Serve static assets only — no Node.js runtime in production	~25 MB image containing Nginx + compressed static files

Key Nginx Configuration Points

- **SPA Fallback Routing:** All routes fall back to `index.html` so React Router handles client-side navigation.
- **Security Headers:** Content Security Policy (CSP), HTTP Strict Transport Security (HSTS), X-Content-Type-Options, and X-Frame-Options are set at the Nginx level.
- **Compression:** Gzip and Brotli pre-compression of JS/CSS bundles for fast initial load.
- **Health Check Endpoint:** Nginx exposes a `/health` endpoint returning HTTP 200, enabling Kubernetes/ECS readiness and liveness probes.
- **Runtime Configuration Injection:** Environment-specific variables (API base URL, SMART on FHIR endpoints, feature flags) are injected at container startup via an `env.js` file generated by the entrypoint script — not baked into the build. This allows the same Docker image to be deployed across development, staging, and production environments.

©Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Container Orchestration Context

The Frontend container is one of several independently deployable services in the Docker Compose / Kubernetes cluster:

Container	Image	Port	Role
`frontend`	`navavej/dashboard:latest`	80/443	React SPA (this section)
`backend-api`	`navavej/api:latest`	3000	NestJS Backend API
`mcp-server`	`navavej/mcp:latest`	3001	MCP Middleware (read-only data access)
`ai-gateway`	`navavej/ai-gateway:latest`	3002	Model-switching AI Gateway
`fhir-db`	`postgres:16-alpine`	5432	PostgreSQL (clinical_fhir + masked schemas)
`trending-db`	`postgres:16-alpine`	5433	Trending Store DB (AI Feature Store)
`auth-server`	`navavej/smart-auth:latest`	8080	SMART on FHIR Authorization Server

Each container has its own `Dockerfile`, health check, resource limits, and can be scaled, replaced, or upgraded independently without affecting other services — directly satisfying TOR §4.3's "Plug/Unplug" requirement.

8.1.6 Non-Blocking UX Pattern (TOR §6.3)

The TOR mandates that the UI must be **non-blocking** — the interface should never freeze while waiting for AI recommendation results. The React architecture achieves this through:

1. **Asynchronous Data Fetching:** All API calls are made via React Query's `useQuery` / `useMutation` hooks, which run in the background. The UI renders immediately with a skeleton/shimmer loading state.
2. **Progressive Rendering:** The Individual Summary screen loads in stages — patient demographics and historical data (from the fast FHIR DB read) render first, while the AI recommendation (which may take up to 10 seconds) loads asynchronously and slots into the page once available.
3. **Optimistic UI for Admin Actions:** Mapping Tool and Master Data changes appear instantly in the UI while the backend processes the request, with automatic rollback if the server rejects the change.
4. **WebSocket/SSE for Real-Time Feeds:** The Access Monitoring Dashboard receives live updates via persistent connections rather than polling, eliminating unnecessary re-renders and network overhead.

8.2 Business Intelligence & Conversion Dashboard (TOR §7)

The TOR (§7) requires a Business Intelligence layer that tracks whether AI-generated health checkup recommendations actually convert into purchased services, and provides an executive-level performance view. This introduces a new stakeholder group — **Management / Executives** — alongside Healthcare Providers and DPO/Admin.

8.2.1 Stakeholders & Access Control

The **Healthcare Provider** role encompasses the full patient consultation workflow — from generating AI recommendations, to presenting and explaining the checkup plan to the patient, to capturing the patient's acceptance or refusal. There is no separate "Sales" role; the provider handles both clinical and consultative responsibilities.

Stakeholder	Role	Dashboard Access
Healthcare Provider	Generates AI recommendations, consults with the patient, presents the checkup plan, captures acceptance or rejection reason	Provider Portal (§7.1.3) + Conversion Tracking UI (this section)
Management / Executive	Reviews business performance, adoption trends, and revenue uplift	Executive Dashboard (this section)

© Copyright 2025 by Vietnam Silicon.

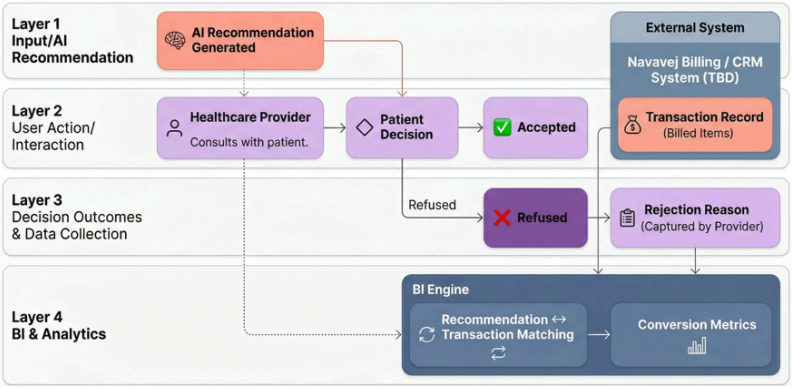
This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

DPO / Admin	Manages access, audit logs, and system configuration	Admin Portal (§7.1.3) — existing

Access to the BI dashboards is role-gated via the existing RBAC framework (§7.1.3, TOR §3.4.4). Management users see aggregated, anonymized metrics only — no access to individual patient records.

8.2.2 Conversion Tracking Workflow

The conversion tracking flow bridges the gap between **AI recommendation** (generated by the system) and **actual purchase** (recorded in Navavej's billing/CRM system):



mermaid

Key Data Points Captured:

Event	Captured By	Data Stored
Recommendation Generated	System (automatic)	Recommendation ID, patient ID (hashed), recommended program, add-on items, confidence score, timestamp
Recommendation Presented	Healthcare Provider (manual)	Presentation timestamp, provider ID
Patient Accepts	Healthcare Provider (manual)	Accepted items (full program, partial add-ons, or custom selection)
Patient Refuses	Healthcare Provider (manual)	Rejection reason from dropdown (see §7.2.3)
Transaction Completed	Billing / CRM system (via integration)	Billed items, transaction amount, transaction date

8.2.3 Rejection Reason Capture

When a patient declines a recommended item, the **Healthcare Provider** captures the reason through a simple dropdown UI integrated directly into the Provider Portal. This is part of the same consultation workflow — after presenting the AI-recommended checkup plan, the provider records the patient's decision and, if refused, the reason.

Rejection Reason Code	Description
`PATIENT_REFUSED`	Patient declined the recommendation
`RECENTLY_EXAMINED`	Patient was recently examined elsewhere
`COST_CONCERN`	Patient cited cost as a barrier
`TIME_CONSTRAINT`	Patient did not have time for the examination
`ALREADY_SCHEDULED`	Patient already has the examination scheduled
`DOCTOR_OVERRIDE`	Attending physician overrode the AI recommendation

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

`OTHER`	Free-text field for unlisted reasons

> **AI Feedback Loop (TOR §7):** Rejection reasons are stored as structured data and fed back into the AI tuning pipeline. Patterns such as "Item X is rejected 60% of the time due to cost" inform prompt adjustments — for example, the AI may learn to prioritize cost-effective alternatives or flag high-cost add-ons with a note for the provider.

8.2.4 External Billing / CRM Integration

Assumption: The specific billing or CRM system to integrate with is **TBD - to be confirmed by Navavej Hospital**. The architecture is designed to support any system that can expose transaction data via API or database-level synchronization.

Integration Aspect	Design Decision
Source System	Navavej billing system, CRM, or IQVIA Billed module — TBD
Integration Method	API polling (preferred) or CDC sync — matching the same pattern used for HIS data ingestion (§4.1)
Matching Logic	Match by `patient_id` + `examination_code` + `date_range` (recommendation date → transaction date within configurable window, e.g., 30 days)
Data Direction	**Read-only** — the Smart Dashboard reads billing data for matching; it never writes to the billing system
Scope	Listed as optional integration in Phase 2 delivery. Core BI dashboards (adoption rate, rejection reasons) function without billing integration; conversion rate requires it.

8.2.5 Executive Dashboard Panels

The Management / Executive dashboard provides aggregated, anonymized business intelligence:

Panel	Metrics Displayed	Source
Conversion Funnel	Recommendations Generated → Presented → Accepted → Billed. Displayed as a funnel chart with conversion % at each stage.	Recommendation log + Billing integration
Uplift Revenue	Total value of add-on examinations generated from AI recommendations (beyond the base program the patient would have selected without AI). Displayed as monthly/quarterly trend.	Billing integration
Adoption Rate	Usage statistics by healthcare provider — which users actively follow AI recommendations vs. override/ignore them. Helps management identify change management needs.	Recommendation log
Top Recommended Items	Most frequently recommended add-on examinations, with acceptance vs. rejection rates.	Recommendation log
Rejection Analysis	Breakdown of rejection reasons by category (cost, time, patient refusal, doctor override). Drill-down by examination type.	Rejection reason log
AI Confidence vs. Conversion	Scatter plot correlating AI confidence scores with actual conversion rates — validates whether higher confidence leads to higher acceptance.	Recommendation log + Billing integration

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

9. Deployment & Operational Architecture

9.1 On-Premise Deployment Strategy

The Smart Health Checkup Recommendation Dashboard is deployed entirely **on-premise** within Navavej Hospital's own data center. This decision is driven by the strict data residency requirements of PDPA, the hospital's preference for full infrastructure ownership, and the elimination of recurring cloud compute costs. All system components run as **Docker containers** orchestrated by Docker Compose on hospital-managed servers, using exclusively open-source software (OSS) with no proprietary cloud service dependencies.

9.1.1 Physical Infrastructure & Hardware Sizing

The system requires a minimum of two physical servers (or VMs) to satisfy basic high-availability and separation-of-concern requirements:

Server Role	Purpose	Recommended Minimum Spec	Containers Hosted
Application Server	Runs all application-tier containers	16 vCPU, 32 GB RAM, 200 GB SSD	`frontend`, `backend-api`, `mcp-server`, `ai-gateway`, `auth-server`, `reverse-proxy`
Database Server	Runs all database and storage containers	8 vCPU, 32 GB RAM, 1 TB SSD (RAID-10)	`fhir-db`, `trending-db`, `audit-db`

> **Scaling Note:** For Year 1 operations (estimated < 50 concurrent users, < 10,000 patient records actively queried per day), a 2 server setup is sufficient. As patient volume grows (Year 2–3), horizontal scaling can be achieved by migrating from Docker Compose to a lightweight Kubernetes distribution (e.g., **K3s**, open-source) or by adding replicated Application Servers behind a load balancer.

Network Prerequisites

- Both servers must reside on the same private network (VLAN) with low-latency connectivity (< 1 ms).
- The Application Server requires outbound HTTPS access (port 443) to external AI provider APIs (OpenAI, Anthropic) — unless using a local LLM exclusively.
- The Database Server should have **no direct outbound internet access** — all external communication is proxied through the Application Server.

9.1.2 Docker Compose Service Architecture

All services are defined in a single `docker-compose.yml` and managed as a unified stack. Each service runs in its own container with explicit resource limits, health checks, and restart policies.

› mermaid

Docker Compose Service Definitions

Service	Image	Port	Restart Policy	Health Check
`reverse-proxy`	`nginx:alpine`	443 → external	`always`	`curl -f https://localhost/health`
`frontend`	`navavej/dashboard:latest`	80 (internal)	`always`	`curl -f http://localhost/health`
`backend-api`	`navavej/api:latest`	3000	`always`	`/api/health` endpoint
`mcp-server`	`navavej/mcp:latest`	3001	`always`	`/health` endpoint
`ai-gateway`	`navavej/ai-gateway:latest`	3002	`always`	`/health` endpoint

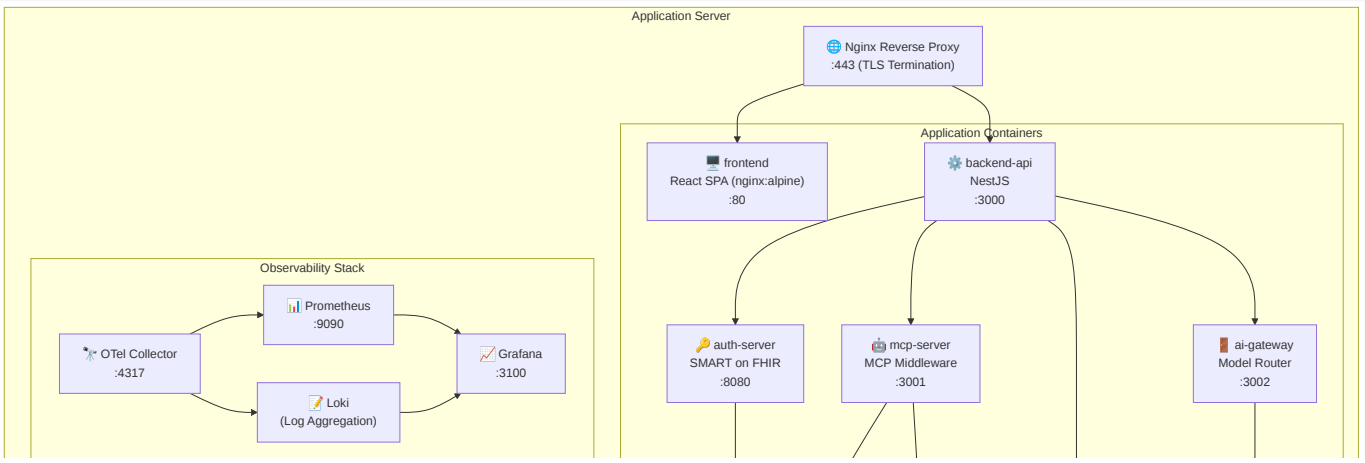
@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

`auth-server`	`navavej/smart-auth:latest`	8080	`always`	`/auth/health` endpoint
`fhir-db`	`postgres:16-alpine`	5432	`always`	`pg_isready`
`trending-db`	`postgres:16-alpine`	5433	`always`	`pg_isready`
`audit-db`	`postgres:16-alpine`	5434	`always`	`pg_isready`
`prometheus`	`prom/prometheus:latest`	9090	`always`	`/-/healthy`
`grafana`	`grafana/grafana-oss:latest`	3100	`always`	`/api/health`
`loki`	`grafana/loki:latest`	3200 (internal)	`always`	`/ready`
`otel-collector`	`otel/opentelemetry-collector:latest`	4317 (gRPC)	`always`	`grpc_health_v1`

9.1.3 Network & Security Topology

All inter-container communication occurs over Docker's internal bridge network. Only the Nginx reverse proxy is exposed to the hospital's internal LAN.



mermaid

Security Hardening

Layer	Implementation
TLS Termination	Nginx handles TLS 1.2/1.3 with hospital-issued certificates. All internal traffic between containers uses Docker's private network (not exposed to LAN).
Firewall Rules	Only port 443 (HTTPS) is open inbound from the hospital LAN. Port 5432–5434 (databases) are never exposed outside the Docker network.
Docker Secrets	Database passwords, API keys (OpenAI/Anthropic), and JWT signing keys are stored in Docker Secrets — never in environment variables or `docker-compose.yml`.

©Copyright 2025 by Vietnam Silicon.

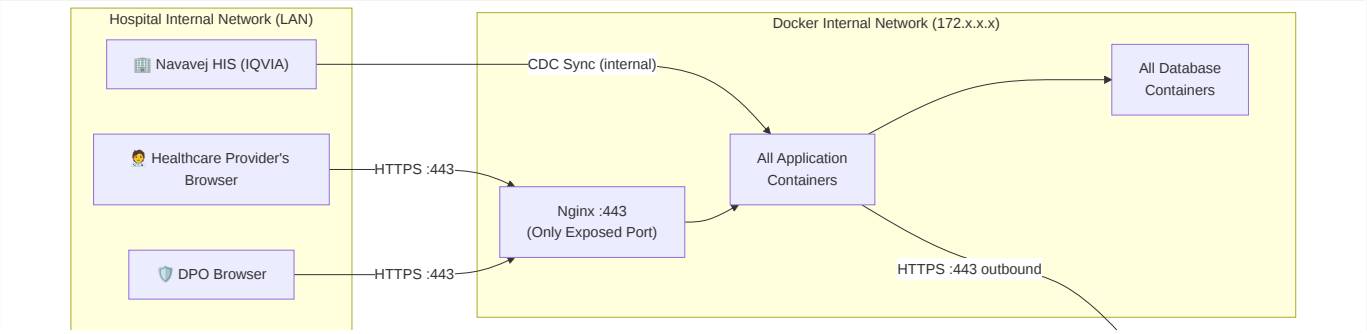
This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Read-Only Filesystems	Application containers (`frontend` , `mcp-server`) run with `read_only: true` Docker flag to prevent runtime file system tampering.
Non-Root Containers	All application containers run as non-root users (`USER 1000`). Database containers use the official PostgreSQL non-root configuration.
Network Isolation	Two Docker networks are defined: `app-net` (application containers) and `db-net` (database containers). Only `backend-api` , `mcp-server` , and `auth-server` are attached to both. The `frontend` and `ai-gateway` cannot reach databases directly.

9.2 Observability & Monitoring (OSS Stack)

The system uses an entirely open-source observability stack built on **OpenTelemetry**, **Prometheus**, **Grafana**, and **Loki** — replacing any cloud-native dependencies (e.g., CloudWatch) to align with the on-premise strategy.

9.2.1 Three Pillars of Observability



mermaid

Pillar	Tool	What It Captures
Metrics	Prometheus	Request latency (p50/p95/p99), error rates, container CPU/RAM usage, DB connection pool status, AI inference duration, queue depth
Logs	Grafana Loki	Structured JSON logs from all services. Correlatable by `trace_id` for end-to-end request debugging. Includes 5-Dimension Audit Logs.
Traces	Grafana Tempo (optional)	Distributed traces across the full request chain: Dashboard → API → MCP → FHIR DB → AI Gateway → LLM. Identifies latency bottlenecks in the < 10s SLA budget.

9.2.2 Pre-Built Grafana Dashboards

The system ships with four pre-configured Grafana dashboards provisioned via `grafana/provisioning/dashboards/` :

Dashboard	Audience	Key Panels
System Health	Ops / Infra Team	Container CPU/RAM/disk usage, Docker restart counts, database connection pool saturation, network I/O
API Performance	Dev / Ops Team	Request volume (RPM), latency percentiles (p50/p95/p99), error rate by endpoint, slowest endpoints
AI Inference Monitor	Dev / Clinical Team	AI inference latency breakdown (MCP fetch time vs. LLM response time), model usage distribution (GPT-4 vs. Claude vs. Llama), token consumption, fallback-to-rules rate

©Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

SLA Compliance	Management / DPO	Real-time SLA tracking: % of requests completing within 10s (new patients) and 30s (existing patients), daily/weekly trends, breach alerts

9.2.3 Alerting

Prometheus Alertmanager is configured to send alerts to the hospital's existing communication channels (e.g., email, LINE Notify, or Teams):

Alert	Condition	Severity
SLA Breach	p95 latency > 10s for 5 consecutive minutes	🔴 Critical
Container Down	Any service health check fails for > 30 seconds	🔴 Critical
Database Disk Usage	Disk usage > 80% on any PostgreSQL volume	🟡 Warning
AI Gateway Error Spike	Error rate > 5% over 5 minutes	🟡 Warning
Masking Pipeline Lag	Pre-masking job backlog > 100 records for > 10 minutes	🟡 Warning
Certificate Expiry	TLS certificate expires within 14 days	🟡 Warning

9.3 Disaster Recovery (DR) & Backup Plan

9.3.1 Backup Strategy

All persistent data is stored on Docker volumes mapped to the Database Server's RAID-10 storage. Backups follow a **3-2-1 rule** (3 copies, 2 different media, 1 offsite) using OSS tooling:

Component	Backup Method	Frequency	Retention	Tool
PostgreSQL Databases (fhir-db, trending-db, audit-db)	`pg_dump` logical backup + WAL archiving for point-in-time recovery	Full daily + WAL continuous	30 days (daily), 7 days (WAL)	`pg_dump`, `pgBackRest` (OSS)
Docker Compose Configuration	Version-controlled in Git	On every change	Indefinite	Git
Grafana Dashboards & Alerts	Provisioned as code (JSON files in Git)	On every change	Indefinite	Git
Docker Volumes (application state)	Filesystem-level snapshot	Daily	7 days	`rsync` or LVM snapshots
Offsite Copy	Encrypted backup transfer to secondary location	Weekly	90 days	`rsync` + GPG encryption over SSH

9.3.2 Recovery Procedures

Scenario	RTO (Recovery Time Objective)	RPO (Recovery Point Objective)	Procedure
Single container crash	< 1 minute	Zero data loss	Docker `restart: always` policy auto-

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

			recovers the container.
Application Server failure	< 1 hour	Zero data loss	Re-deploy `docker-compose up -d` on a standby server using the same images and configuration from Git. Database is on a separate server and unaffected.
Database Server failure	< 4 hours	< 24 hours (daily backup) or < 5 minutes (WAL recovery)	Restore from `pgBackRest` backup to a replacement server. Replay WAL logs for point-in-time recovery.
Full site disaster	< 8 hours	< 1 week (offsite backup)	Provision new servers, restore from offsite encrypted backup, re-deploy all containers.

9.3.3 Data Integrity & Immutability

- **Audit Log Immutability:** The `audit-db` PostgreSQL instance enforces immutability through database-level triggers that prevent `UPDATE` and `DELETE` operations on audit tables. The database user for the `backend-api` service has only `INSERT` and `SELECT` privileges on audit tables — no `UPDATE`, `DELETE`, or `TRUNCATE`.
- **FHIR Record Integrity:** The `clinical_fhir` schema uses append-only patterns for clinical records. Historical records are never modified; corrections are stored as new FHIR resources with references to the original (per FHIR versioning semantics).
- **Backup Verification:** Automated weekly test restores are executed against a temporary PostgreSQL instance to verify backup integrity. Results are logged and surfaced on the System Health Grafana dashboard.

10. Quality Assurance & Non-Functional Requirements (NFRs)

10.1 Performance Metrics & SLAs

The TOR (§6.3) defines two distinct performance tiers based on whether the patient's data has already been ingested and pre-processed (existing customer) or requires first-time extraction from the HIS (new customer). The system is designed to meet both tiers through the architectural decisions described in prior sections — namely pre-masked data (§5.2), pre-computed trending features (§4.2.3), and non-blocking progressive UI rendering (§7.1.6).

10.1.1 Latency SLA Targets

Metric	New Patient (First Visit)	Existing Patient (Return Visit)
Data Retrieval (MCP → masked FHIR + Trending DB)	< 3 seconds	< 10 seconds
AI Recommendation Processing (AI Gateway → Gemini → Hybrid Engine)	< 7 seconds	< 20 seconds
Total End-to-End (button click → recommendation displayed)	**≤ 10 seconds**	**≤ 30 seconds**
UI Responsiveness	Non-blocking — skeleton/shimmer rendering while AI processes	Non-blocking — identical progressive rendering behavior

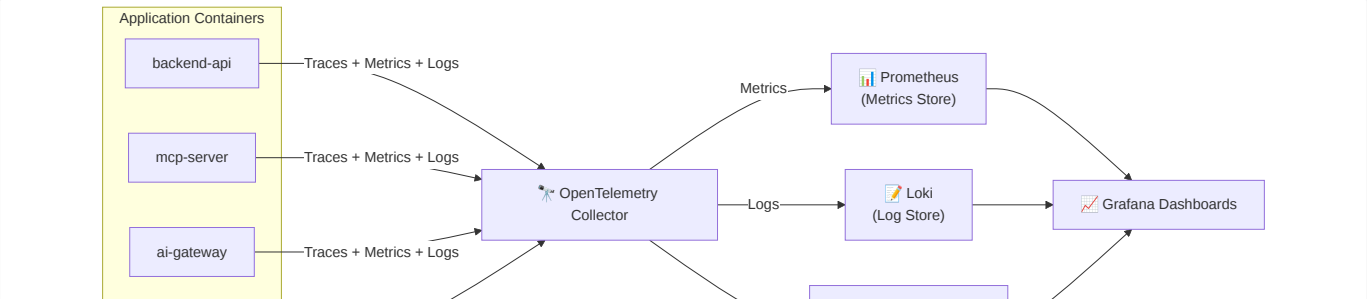
@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

> **Why existing patients may take longer:** Existing patients have significantly larger clinical histories (3–5 years of longitudinal data, lab trends, screening records). The MCP must fetch and assemble a larger masked context payload, and the AI model must process more input tokens to generate an accurate recommendation.

10.1.2 Latency Budget Breakdown

To ensure the SLA is achievable, the following latency budget allocates time across each component in the critical path:



mermaid

Component	New Patient Budget	Existing Patient Budget
SMART on FHIR token validation	~200 ms	~200 ms
MCP data fetch (masked schema)	~1.5 s	~8 s
Freshness-gap check (§5.2.2)	~300 ms	~300 ms
Pre-AI safety rules (Hybrid Engine)	~500 ms	~500 ms
AI Gateway → Gemini 1.5 Pro inference	~5 s	~16 s
Post-AI safety rules validation	~500 ms	~500 ms
Frontend rendering	~500 ms	~500 ms
Buffer / variance	~1.5 s	~3.5 s
Total	**≤ 10 s**	**≤ 30 s**

10.1.3 Performance Testing Strategy

Test Type	Tool	Methodology
Load Testing	k6 (OSS) or Apache JMeter	Simulate 30 concurrent users each triggering 5 recommendation requests within 10 minutes. Measure p50, p95, p99 latency.
Stress Testing	k6	Gradually increase concurrent users from 30 → 60 → 100 to identify the breaking point and validate graceful degradation.
Soak Testing	k6	Run 30 concurrent users for 8 consecutive hours to detect memory leaks, connection pool exhaustion, or degraded AI response times.
AI Latency Profiling	OpenTelemetry traces	Instrument each component in the critical path to identify bottlenecks. Dashboard available in Grafana (§8.2.2 — AI Inference Monitor).

SLA Compliance Monitoring: The Grafana SLA Compliance dashboard (§8.2.2) continuously tracks the percentage of requests completing within the defined SLA targets. Prometheus Alertmanager triggers a Critical alert if p95 latency exceeds 10s for new patients or 30s for existing patients for 5 consecutive minutes.

© Copyright 2025 by Vietnam Silicon

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

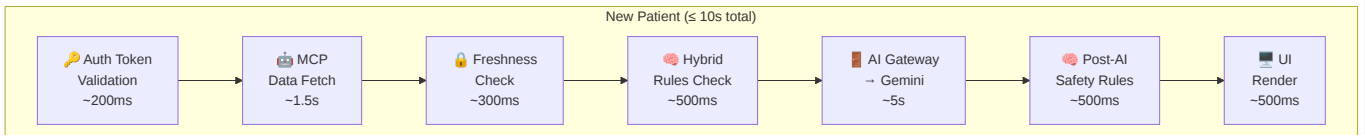
10.2 Scalability & High Availability Matrix

NFR Dimension	Target	How Achieved
Concurrent Users	30 users at peak (Year 1)	Single Application Server (16 vCPU, 32 GB) handles all application containers. NestJS backend uses async/non-blocking I/O.
Horizontal Scaling	Up to 100 users (Year 2–3)	Migrate from Docker Compose to K3s (lightweight Kubernetes). Replicate `backend-api` and `mcp-server` behind a load balancer.
Database Scaling	10,000 → 50,000 patients	PostgreSQL connection pooling (PgBouncer), read replicas for the `masked_clinical_fhir` schema, and table partitioning by patient ID range.
AI Throughput	900 prompts/day (Year 1) → 3,000+ (Year 3)	AI Gateway supports concurrent model calls. Gemini 1.5 Pro API scales automatically. For higher throughput, enable request batching or add a second model endpoint.
Availability Target	99.5% uptime (business hours)	Docker `restart: always` for auto-recovery. Database on separate server (fault isolation). Grafana alerts for immediate incident detection.
Data Storage Growth	~12 GB Year 1 → ~50 GB Year 3	1 TB RAID-10 on DB server provides ~20× headroom. Disk usage alerts at 80% (§8.2.3).

10.3 Medical Validation Strategy (Medical Acceptance Test)

The TOR (§6.2) mandates that the AI recommendation engine must achieve **accuracy comparable to medical experts at > 95%**, validated through a structured blind test protocol. This section defines the complete MAT framework.

10.3.1 Blind Test Protocol



› mermaid

10.3.2 Test Case Design

Dimension	Requirement
Minimum Cases	≥ 100 patient cases (per TOR §6.2)
Diversity Coverage	Cases must span: male/female, ages 20–80, low/moderate/high risk profiles, patients with comorbidities (DM, hypertension), patients with genetic risk factors, patients with behavioral risk factors (smoking, alcohol)
Data Blinding	AI system and human experts receive identical de-identified clinical data. Neither party knows the other's recommendations during evaluation.
Expert Panel	5 independent physicians. Specialists should include: internal medicine, preventive medicine, and at least one sub-specialist (cardiology, endocrinology, or oncology).

©Copyright 2025 by Vietnam Silicon

10.3.3 Scoring Methodology

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Metric	Definition	Pass Threshold
Expert Consensus (Gold Standard)	A recommendation is considered the "correct" answer when **at least 3 out of 5** experts agree on it independently (per TOR §6.2).	—
Primary Program Accuracy	% of cases where the AI's recommended primary program (Standard / Advanced / Premium) matches the expert consensus.	**≥ 95%**
Add-On Recommendation Accuracy	% of cases where the AI's add-on examination list matches expert consensus (measured by F1 score — precision + recall).	**≥ 90%**
Critical Safety Compliance	% of cases where the AI does NOT violate any Safety Rule (e.g., gender-inappropriate screening).	**100%** (zero tolerance)
Screening Gap Detection	% of known screening gaps correctly identified by the AI compared to expert assessment.	**≥ 90%**

10.3.4 MAT Execution Timeline

Phase	Duration	Activities
Preparation	2 weeks	Case selection, expert panel recruitment, data de-identification, test environment setup
Expert Review	2 weeks	Each expert independently reviews all 100+ cases and submits recommendations
AI Execution	1 day	Run all 100+ cases through the production-equivalent AI pipeline and record outputs
Scoring & Analysis	1 week	Compare results, calculate metrics, identify failure patterns
Remediation (if needed)	2 weeks	Adjust safety rules, retune prompts, re-run failed cases
Final Report	1 week	Compile MAT Report documenting methodology, results, and expert sign-off

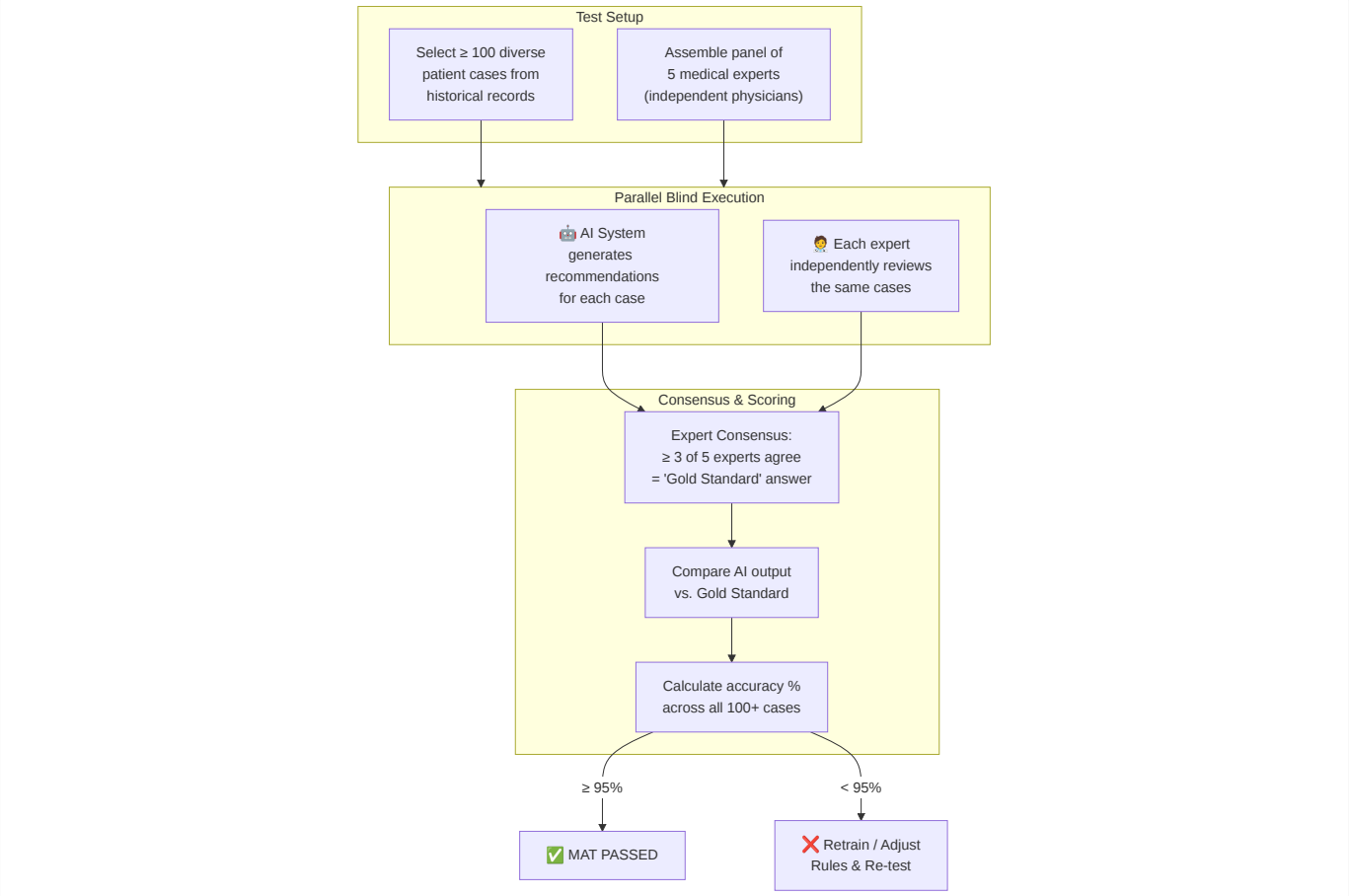
10.4 Safety "Fallback" Rules

The TOR (§6.2) requires that the system include configurable Safety Rules and a Safety Fallback mechanism to ensure that AI recommendations never compromise patient safety. The Hybrid Recommendation Engine (§3.2, item 5) implements this through a **three-layer safety architecture**:

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

10.4.1 Three-Layer Safety Architecture



mermaid

10.4.2 Safety Rule Categories

All safety rules are **configurable via the Master Data Management UI** (§7.1.3, TOR §3.4.6) without requiring code changes. Rules are version-tracked and require maker-checker approval before activation.

Rule Category	Example Rules	Source
Gender Restriction	Mammogram → female only; Prostate screening → male only	TOR §6.2
Age Limits	Colonoscopy screening → age ≥ 50; Pediatric panels → age < 18 excluded	Clinical guidelines
Contraindication Check	Do not recommend contrast CT if patient has severe kidney impairment (eGFR < 30)	Clinical safety
Pregnancy Safety	Exclude X-ray and CT-based examinations for pregnant patients	Clinical safety
Frequency Cap	Do not recommend Mammogram if one was performed < 12 months ago	Screening guidelines
Drug Interaction	Flag if recommended lab tests conflict with active medications	Clinical safety

10.4.3 Fallback Behavior Matrix

Scenario	System Behavior	UI Indicator
AI succeeds, no safety conflicts	Full AI recommendation with Explainable AI reasoning and	✅ Normal display

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

	Confidence Score	
AI succeeds, but conflicts with Safety Rule	Conflicting items are **revoked** and logged. Remaining safe items are displayed. Revoked items are listed with the reason.	⚠️ Yellow badge: "Some items adjusted by safety rules"
AI times out (> SLA threshold)	Hybrid Engine falls back to **rule-based only** recommendation. No AI reasoning or Confidence Score available.	🟠 Orange badge: "Rule-based recommendation (AI unavailable)"
AI returns error / malformed response	Same as timeout — full fallback to rule-based engine	🟠 Orange badge
Both AI and rule engine fail	Display patient summary data only (trends, screening gaps). No program recommendation generated. Log critical alert.	🔴 Red badge: "Unable to generate recommendation — contact support"

10.4.4 Circuit-Breaker Pattern

To prevent cascading failures when the external AI API (Gemini) is experiencing an outage, the AI Gateway implements a **circuit-breaker**:

State	Condition	Behavior
Closed (normal)	Error rate < 5%	All requests sent to Gemini normally
Open (tripped)	Error rate ≥ 5% over 5 minutes, or 3 consecutive timeouts	All requests immediately routed to rule-based fallback — **no calls to Gemini**. Reduces latency from ~10s (waiting for timeout) to ~1s (instant rule-based response).
Half-Open (probing)	After 60 seconds in Open state	Send 1 test request to Gemini. If successful → return to Closed. If failed → remain Open for another 60s.

This pattern ensures that when Gemini is down, users still receive instant rule-based recommendations rather than experiencing repeated 10-second timeouts. The circuit state is visible on the AI Inference Monitor Grafana dashboard (§9.2.2).

11. Infrastructure Licensing & Cloud Costs

11.1 Workload Assumptions

> **Important:** All cost estimates in this section are calculated for the following specific workload profile. If the actual workload differs significantly (e.g., higher concurrency, more AI calls, or a larger patient base), infrastructure sizing and costs must be re-evaluated accordingly.

Parameter	Assumed Value
Total Healthcare Providers	30 users in total
Concurrent Users	30 healthcare providers using the dashboard simultaneously (peak)
AI Recommendation Prompts	30 prompts per user per day × 30 users = **900 prompts/day** (Gemini 1.5 Pro API calls)
Total Patient Records	10,000 patients with 3–5 years of historical data each
Data Volume (estimated)	~5 GB raw FHIR data + ~5 GB masked copy + ~2 GB trending store = ~12 GB total

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

AI Context Size per Prompt	~4,000 input tokens (masked clinical context + prompt template)
AI Response Size per Prompt	~1,500 output tokens (recommendation + reasoning + confidence score)
Operating Hours	Business hours (10 hrs/day), 5 days/week, ~22 working days/month

11.2 Software Licensing Costs

The entire software stack is built on **open-source software (OSS)** with permissive licenses. There are **zero recurring software licensing fees**:

Component	Software	License	Cost
Frontend	React, Vite, TypeScript	MIT	**$0**
Backend API	NestJS, Node.js	MIT	**$0**
Database	PostgreSQL 16	PostgreSQL License (permissive)	**$0**
AI Framework	LangChain / Custom Gateway	MIT	**$0**
Auth Server	Keycloak / SMART on FHIR OSS	Apache 2.0	**$0**
Data Masking	Microsoft Presidio, PyThaiNLP	MIT	**$0**
Observability	Prometheus, Grafana OSS, Loki, OpenTelemetry	Apache 2.0	**$0**
Containerization	Docker Engine (Community)	Apache 2.0	**$0**
Orchestration	Docker Compose / K3s	Apache 2.0	**$0**
Backup	pgBackRest	MIT	**$0**
Total Software Licensing			**$0 / month**

11.3 External AI API Costs (Gemini 1.5 Pro - example)

Regardless of on-premise or cloud infrastructure, the external AI API cost is the same — it depends solely on the number of prompts and token volumes:

Metric	Calculation	Value
Daily Input Tokens	900 prompts × 4,000 tokens	3,600,000 tokens/day
Daily Output Tokens	900 prompts × 1,500 tokens	1,350,000 tokens/day
Monthly Input Tokens	3,600,000 × 22 working days	79,200,000 tokens/month
Monthly Output Tokens	1,350,000 × 22 working days	29,700,000 tokens/month

@Copyright 2025 by Vietnam Silicon.

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Token Type	Volume / Month	Gemini 1.5 Pro Rate	Monthly Cost
Input Tokens	79.2M tokens	$1.25 / 1M tokens	**$99.00**
Output Tokens	29.7M tokens	$5.00 / 1M tokens	**$148.50**
Total AI API Cost			**~$248 / month**
Annual AI API Cost			**~$2,970 / year**

> **Note:** At 900 prompts/day, AI API costs are a moderate but manageable expense (~$248/month). This cost scales linearly with user count — if providers double to 60 users, AI cost would approximately double to ~$496/month. Gemini 1.5 Pro's competitive token pricing keeps this significantly lower than equivalent models (e.g., GPT-4 Turbo at ~3× this rate).

11.4 Infrastructure Cost Comparison: On-Premise vs. AWS Cloud

Below is a side-by-side comparison of the two deployment options. Both options run the identical Docker container architecture — only the underlying compute and storage layer differs.

Option A: On-Premise (Hospital Data Center)

Item	Specification	One-Time Cost (USD)	Monthly Cost (USD)
Application Server	16 vCPU, 32 GB RAM, 200 GB SSD (e.g., Dell PowerEdge R350 or equivalent)	~$3,000 – $4,500	$0 (owned)
Database Server	8 vCPU, 32 GB RAM, 1 TB SSD RAID-10 (e.g., Dell PowerEdge R350 or equivalent)	~$3,500 – $5,000	$0 (owned)
Network Switch / Firewall	Managed switch + existing hospital firewall	~$500 – $1,000	$0
UPS / Power Protection	Rack-mounted UPS for both servers	~$800 – $1,500	$0
TLS Certificate	Hospital-issued or Let's Encrypt	$0	$0
Electricity & Cooling	~0.5 kW × 2 servers, 24/7	—	~$50 – $80
Offsite Backup Storage	External NAS or secondary location	~$500 – $1,000	$0
Gemini 1.5 Pro API	1,500 prompts/day (see §11.1.3)	—	~$248
Total (On-Premise)		**~$8,300 – $13,000** (one-time)	**~$298 – $328 / month**

Option B: AWS Cloud

@Copyright 2025 by Vietnam Silicon.

Item	AWS Service	Specification	Monthly Cost (USD)

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Application Server	EC2 `t3.xlarge` (4 vCPU, 16 GB)	1 instance × On-Demand	~$120
Database Server	RDS PostgreSQL `db.t3.large` (2 vCPU, 8 GB)	Multi-AZ NOT required at this scale; single instance	~$100
— or —	EC2 `t3.large` (2 vCPU, 8 GB) + self-managed PostgreSQL on Docker	Lower cost, same as on-prem architecture	~$60
Storage (EBS)	gp3 SSD	200 GB (App) + 500 GB (DB)	~$56
Data Transfer	Outbound to Internet	~5 GB/month (AI API calls + frontend assets)	~$0.45
Backup	AWS Backup / S3 Standard	50 GB daily snapshots, 30-day retention	~$5
TLS Certificate	ACM (AWS Certificate Manager)	Free with ALB/CloudFront	$0
Load Balancer (optional)	ALB	For TLS termination + routing	~$22
Gemini 1.5 Pro API	Google AI API (external to AWS)	900 prompts/day (see §11.1.3)	~$248
Total (AWS — EC2 + self-managed DB)			**~$511 / month**
Total (AWS — EC2 + RDS managed DB)			**~$551 / month**

> **Note on AWS Cost Optimization:** If Navavej commits to a **1-year Reserved Instance** plan (no upfront), infra costs reduce by ~30%, bringing the monthly total to approximately **~$430 – $470/month** (including AI API). A **3-year RI** commitment reduces infra costs by ~50%.

11.5 Three-Year Total Cost of Ownership (TCO) Summary

Cost Category	On-Premise	AWS Cloud (On-Demand)	AWS Cloud (1-Year RI)
Year 0: One-Time Hardware	$10,000 (avg)	$0	$0
Year 1: Monthly Infra	$70 × 12 = $840	$290 × 12 = $3,480	$205 × 12 = $2,460
Year 1: AI API	$2,970	$2,970	$2,970
Year 1: Software Licensing	$0	$0	$0
Year 1 Total	**$13,810**	**$6,450**	**$5,430**
Year 2 Total	**$3,810**	**$6,450**	**$5,430**
Year 3 Total	**$3,810**	**$6,450**	**$5,430**
3-Year TCO	**$21,430**	**$19,350**	**$16,290**

@Copyright 2025 by Vietnam Silicon.

11.6 Comparison Summary & Recommendation

This document is confidential and restricted to Vietnam Silicon use only; it cannot be shared with third parties without prior written authorization from Vietnam Silicon

Factor	🏠 On-Premise	☁️ AWS Cloud
Year 1 Cost	Higher ($13.8K — front-loaded hardware + AI API)	Lower ($5.4K – $6.5K)
3-Year TCO	$21.4K (hardware amortizes, but AI API is ongoing)	$16.3K – $19.4K (ongoing monthly costs)
AI API % of Total Cost	~14% of 3-year TCO	~15–18% of 3-year TCO
Data Residency	✅ Full PDPA control — data never leaves hospital premises	⚠️ Requires AWS Bangkok Region (ap-southeast-1) and PDPA data processing agreement
Operational Effort	⚠️ Hospital IT team manages hardware, patching, physical security	✅ AWS manages physical infra, patching, power — hospital manages only Docker containers
Scalability	⚠️ Requires additional hardware purchase for scaling beyond Year 1 capacity	✅ Scale up/down instantly by changing instance type
DR & Availability	⚠️ Single-site — requires offsite backup discipline	✅ Multi-AZ, automated backups, easier disaster recovery
Network Latency to AI API	Depends on hospital's internet connection	Low latency from AWS to Google AI API (cloud-to-cloud)

> **Recommendation:** For a workload of 30 healthcare providers generating 900 AI prompts/day, both options are financially viable with comparable 3-year TCOs ($16K – $21K). The **AI API cost ($248/month)** represents ~14–18% of total spend — moderate but manageable. **On-Premise** is best suited if Navavej Hospital has an existing data center with IT operations staff and prioritizes absolute data residency control. **AWS Cloud** is best suited if the hospital prefers minimal operational overhead and values elastic scalability for future growth. In either scenario, the infrastructure decision should be driven by data residency and operational preferences — the AI API cost is identical regardless of hosting.

@Copyright 2025 by Vietnam Silicon.